Inhalt

1 Einleitung....3
1.1 Vorwort....3
1.2 Abstract - Deutsch....4
1.3 Abstract - Italiano....5
1.4 Abstract - Englisch....6
1.5 Idee/Inspiration:....7
1.6 Konzept für Umsetzung....8
1.7 Meilensteine & Zeitplanung....9
1.8 Projektantrag....10
1.9 Pflichtenheft....11
2 Wahl der Bauteile....12
2.1 Microcontroller....12
2.2 GPS Modul....13
2.3 Temperatursensor....13
2.4 Display....14
2.5 Batterie....15
2.7 Spannungsversorgung....16
3 Schaltung....18
3.1 Blockschaltbild....18
3.2 Spannungsversorgung/Laderegler....19
3.3 USB Schnittstelle....20
3.4 Microcontroller....21
4 Platine....23
4.1 Spannungsversorgung....23
4.2 USB Schnittstelle....24
4.3 Microcontroller & Sensorik....25
4.4 Vorgenommene Änderungen....26
5 Software....28
5.1 Aufbau....28
5.2 GPS Modul....29
5.3 Temperatursensor....30
5.4 Display....31
5.5 Batteriestand....33

6 Gehäuse.....34
6.1 Voraussetzungen.....34
6.2 Design & Umsetzung.....35
6.3 Ergebnis:.....36
6.4 Einbau in Skibrille.....38
6.5 Optik.....40
7 Projektmanagement.....41
7.1 Wochenberichte.....41
7.2 Beraterstunden.....42
7.3 Kostenberechnung.....43
8 Fazit.....45
8.1 Selbstreflektion.....45
8.2 Schlussfolgerung.....46
9 Anhänge.....47
9.1 Projektantrag.....47
9.2 Pflichtenheft.....49
9.3 Gantt- Diagramm.....50
9.4 Wochenbericht.....53
9.5 Schematik.....54
9.6 Board.....56
9.7 Software.....57
9.8 Quellen.....62
9.9 Copyright.....63

1 Einleitung

1.1 Vorwort

Sehr geehrter Leser,

mein Name ist Lorenz Alber und auf den folgenden Seiten präsentiere ich die Dokumentation meines Maturaprojekts an der TFO Meran für das Schuljahr 2024/25. Ziel dieses Projekts war es, einen funktionalen Prototypen zu entwerfen und umzusetzen. Dabei sollten wir das in den vergangenen Jahren erworbene Wissen in den Bereichen Elektronik, Automation und TPS anwenden und vertiefen.

Eine zentrale Vorgabe war es, ein eigenes Projekt von Grund auf zu planen, eine dazugehörige Platine selbst zu entwerfen und zu zeichnen sowie einen Microcontroller einzubinden. Diese Hardware sollte anschließend programmiert und erfolgreich in Betrieb genommen werden.

Nach sorgfältiger Überlegung entschied ich mich, eine handelsübliche Skibrille mit digitaler Anzeige zu erweitern. Mein Ziel war es, eine kompakte Anzeige zu integrieren, die wichtige Informationen wie Geschwindigkeit und Temperatur in Echtzeit darstellt.

Dieses Projekt bot jedem Schüler die Gelegenheit, seine individuellen Stärken und Schwächen besser kennenzulernen und herauszufinden, welche technischen Bereiche besonders interessieren und Freude bereiten. Durch die eigenständige Arbeit an einem vollständigen Entwicklungsprozess – von der Idee über die Planung und Umsetzung bis hin zur Präsentation – konnten wir nicht nur unsere fachlichen Kompetenzen weiter ausbauen, sondern auch wertvolle Erfahrungen im Projektmanagement sammeln.

Ich lade Sie herzlich ein, mich auf den nächsten Seiten auf meinem Entwicklungsweg zu begleiten und mehr über die einzelnen Schritte, Herausforderungen und Lösungen meines Projekts zu erfahren.

1.2 Abstract - Deutsch

Die Idee für mein Maturaprojekt entstand aus dem Wunsch, beim Skifahren nützliche Informationen wie Geschwindigkeit und Temperatur jederzeit im Blick zu haben. Also setzte ich mir das Ziel, eine Skibrille speziell für Wintersportler zu entwickeln, die während der Fahrt diese Daten anzeigt. Dazu wird ein transparenter Monitor direkt in das Glas der Brille integriert, der die Daten gut sichtbar darstellt, ohne die Sicht zu beeinträchtigen.

Ein Kabel führt vom Display zur Rückseite des Helms, wo eine kompakte Box angebracht ist. In dieser Box sind alle nötigen Sensoren und die Elektronik untergebracht, um die Daten zuverlässig zu erfassen und auszuwerten. Das gesamte System wird von einer Batterie versorgt, die so ausgelegt ist, dass sie einen ganzen Skitag problemlos durchhält.

Im Projekt habe ich zentrale Themen wie Elektronik, Sensorik, Programmierung, Projektmanagement und Produktdesign praktisch umgesetzt und dabei nicht nur mein Fachwissen erweitert, sondern auch viel über meine eigenen Fähigkeiten gelernt.

1.3 Abstract - Italiano

L'idea per il mio progetto di maturità è nata dal desiderio di avere informazioni utili come la velocità e la temperatura sempre a portata di mano mentre si scia. Così mi sono posto l'obiettivo di sviluppare una maschera da sci specificamente per gli sportivi invernali che visualizza questi dati durante la discesa. A tal fine, un monitor trasparente è integrato direttamente nella lente della maschera, mostrando i dati in modo chiaro senza compromettere la visibilità.

Un cavo collega il display alla parte posteriore del casco, dove è montato un piccolo box. In questo box sono contenuti tutti i sensori e l'elettronica necessari per raccogliere ed elaborare i dati in modo affidabile. L'intero sistema è alimentato da una batteria progettata per durare un'intera giornata di sci senza problemi.

Nel progetto ho affrontato temi centrali come elettronica, sensori, programmazione, gestione del progetto e design del prodotto, mettendo in pratica le mie conoscenze e imparando molto anche sulle mie capacità personali.

1.4 Abstract - Englisch

The idea for my graduation project came from the desire to have useful information like speed and temperature always in view while skiing. So, I set myself the goal of developing a ski goggle specifically for winter sports enthusiasts that displays this data during the ride. To achieve this, a transparent monitor is integrated directly into the goggle lens, showing the data clearly without affecting visibility.

A cable runs from the display to the back of the helmet, where a compact box is mounted. This box contains all the necessary sensors and electronics to reliably collect and process the data. The entire system is powered by a battery designed to last an entire day of skiing without any issues.

In the project, I practically implemented key topics such as electronics, sensors, programming, project management, and product design, which not only expanded my technical knowledge but also taught me a lot about my own abilities.

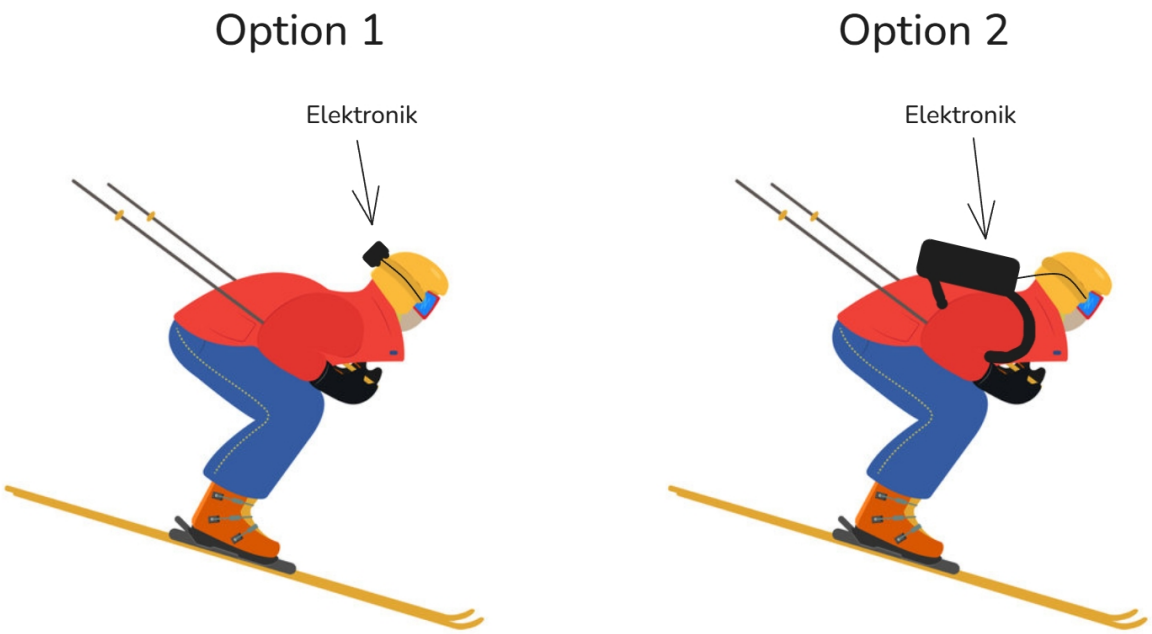
1.5 Idee/Inspiration:

Viele Skifahrer kennen dieses Problem. Um beim Skifahren den Überblick über Top Speed und Geschwindigkeiten zu erhalten, benötigt man eine Tracker App auf dem Smartphone. Dieses ist während der Fahrt jedoch nicht bedienbar und man kann erst am Ende des Skags einen Blick auf die Daten werfen. Auch wenn man auf der Piste wissen will, wie spät es gerade ist, muss man sein Smartphone zücken, um die Uhrzeit abzulesen. Auch das ist lästig. Dieser “Informationsmangel” auf der Piste störte mich schon immer beim Skifahren. Im Frühjahr 2024 kam mir jedoch eine Idee, wie man dieses Problem lösen kann. Ich saß vor dem Fernseher und zappte durch die Sender und stieß plötzlich auf den Film Ironman. Der Superheld trug einen eisernen Anzug und eine Maske, in der die Telemetriedaten auf die Scheibe der Maske projiziert wurden. In mir machte es Klick und ich fragte mich, ob ein ähnliches Konzept auch in der Realität funktionieren würde, um zum Beispiel meine Skifahr- Probleme zu lösen. Nach einigen Recherchen bei Google war ich überrascht über die hohen Preise. Die günstigste Skibrille mit einem Head-up-Display startete bei 400€. Für mich kam so etwas nicht in Frage. Nach einer kurzen Brainstorming Session beschloss ich, dieses Projekt selbst umzusetzen.



1.6 Konzept für Umsetzung

Das Herzstück meines Projektes sollte eine Platine mit Microcontroller sein, die in einer Box an der Rückseite eines Skihelms befestigt oder im Rucksack des Skifahrers verstaut werden sollte. Sie wird mit einer Batterie versorgt und kann einfach mit einem Kippschalter ein- oder ausgeschaltet werden. An dieser Platine wird mit einem GPS-Modul die Geschwindigkeit und Uhrzeit des Skifahrers gemessen. Ein Temperatursensor, der sich ebenfalls auf der Platine befindet, wertet die Umgebungstemperatur aus. Um den Batteriestand auszuwerten, greife ich die Klemmenspannung auf einen Analogpin des Mikrocontrollers ab. All diese Daten werden nun über I2C oder SPI auf ein transparentes Display übertragen und angezeigt, der im Glas der Skibrille verbaut ist. Eine davor montierte Linse sorgt für eine optische Vergrößerung und bessere Lesbarkeit der Anzeige.



1.7 Meilensteine & Zeitplanung

Um meine Meilensteine festzulegen, musste ich mir zunächst eigene Ziele setzen und abschätzen, wie viel Zeit ich für jedes Arbeitspaket benötigen würde.

Zu Beginn definierte ich mein Projekt und erstellte ein Pflichtenheft, in dem alle Anforderungen und Ziele für die Bearbeitung sowie das fertige Produkt festgehalten sind. Für diese Phase plante ich drei Wochen ein.

Anschließend begann ich mit der Planung der Schaltung, wofür ich ebenfalls drei Wochen berechnete. Für das Zeichnen der Platine rechnete ich mit einem Zeitraum von sechs Wochen, sodass ich sie bis spätestens 25. November 2024 bestellt haben musste.

Während ich auf die Lieferung der Platine nach Meran wartete, plante ich etwa drei Wochen für den Bau des Gehäuses ein. Sobald die Platine eingetroffen war, wollte ich mit dem Bestücken beginnen, wofür ich die restliche Zeit bis Weihnachten vorsah.

Nach den Ferien habe ich einen Puffer von rund vier Wochen eingeplant, um mögliche Fehler auf der Platine zu beheben. Danach widmete ich mich der Softwareentwicklung. Bis zum 17. Januar sollte die Displayanzeige vollständig funktionieren. Im Anschluss nahm ich mir drei Wochen für die Auswertung der Sensordaten Zeit, gefolgt von vier Wochen, um alle Softwarekomponenten zusammenzuführen.

In den letzten Wochen des Projekts arbeitete ich parallel zur Programmierung weiter am Gehäuse, sodass das gesamte Projekt bis zum 6. April 2025 abgeschlossen werden konnte.

Parallel zur praktischen Umsetzung des Projekts waren wir Schüler auch verpflichtet, wöchentlich einen Bericht über unsere Fortschritte zu verfassen. Zusätzlich führten wir eine Liste der sogenannten „Beraterstunden“, in denen wir Unterstützung durch Mitschüler, Lehrpersonen oder externe Helfer in Anspruch nahmen.

Die Dokumente dazu finden sie in den Anhängen.

1.8 Projektantrag

Bevor mit der eigentlichen Umsetzung meines Maturaprojekts begonnen werden konnte, musste ein Projektantrag bei den Fachlehrern eingereicht werden. Dieser Schritt markierte den offiziellen Start in die Realisierungsphase des Projekts und war entscheidend dafür, ob und in welcher Form das Vorhaben genehmigt wurde.

Im Antrag wurden die Projektidee, der geplante Ablauf mit den wichtigsten Meilensteinen sowie eine genaue Definition des erwarteten Endergebnisses festgehalten. Letzteres ist besonders wichtig für die spätere Bewertung des Projekts, da hier klar ersichtlich sein muss, welche Funktionen das Produkt am Ende erfüllen soll.

Nach der Einreichung besprachen die Lehrer gemeinsam mit uns Schülern, ob und wie das Projekt in der vorgesehenen Zeit und mit den zur Verfügung stehenden Mitteln umsetzbar ist. Erst nach einer finalen Abstimmung und Freigabe durch die Lehrpersonen galt das Projekt als offiziell genehmigt.

1.9 Pflichtenheft

Nach der Genehmigung des Projektantrags wurde ein Pflichtenheft erstellt. Es dokumentiert die grundlegenden technischen Anforderungen und legt fest, welche Funktionen und Komponenten das Projekt umfassen muss. Das Pflichtenheft dient während der Umsetzung als klare Orientierung und Referenz für die Zielerreichung.

Im Pflichtenheft sind die folgenden Punkte konkret festgelegt:

- Projektziel: Das System soll ein Anzeigeinstrument für eine Skibrille werden. Es misst die Geschwindigkeit mittels GPS-Sensor und zusätzlich die Außentemperatur. Außerdem zeigt es Uhrzeit und Batteriestand auf einem transparenten OLED-Display an, das direkt in die Skibrille eingebaut ist.

- Spannungsversorgung (PSU): Die Stromversorgung erfolgt über eine 3,7V-Flach-LiPo-Batterie, die per USB-C geladen wird. Ein Step-Up-Konverter wandelt die Spannung auf 5V, und ein Schalter erlaubt das Ein- und Ausschalten der gesamten Elektronik.

- Mikrocontroller: Zum Einsatz kommt ein ATMEGA324, programmiert über das Arduino-Framework. Der Batteriestand wird über einen Spannungsteiler erfasst und am Mikrocontroller ausgewertet.

- Sensorik: Verwendet wird ein GPS-Sensor zur Ermittlung der Geschwindigkeit anhand von Koordinatenveränderungen. Ein zusätzlicher Temperatursensor misst die Umgebungstemperatur.

- Display: Das verwendete OLED-Display wird über I2C angesteuert. Dabei soll keine Library verwendet werden.

- Gehäuse: Die Platine und alle Komponenten, außer dem Display und dem Temperatursensor, werden in einer Box im Rucksack untergebracht. Das Display in der Skibrille sowie der Temperatursensor sind über längere Kabel mit der Box verbunden, um während der Fahrt genaue Daten anzeigen zu können.

Das Pflichtenheft wurde nun von Herrn De Tomaso und Huber unterzeichnet und mit dem Prüfvermerk „OK“ bestätigt. Damit war es die verbindliche Grundlage für die Projektumsetzung und spätere Bewertung.

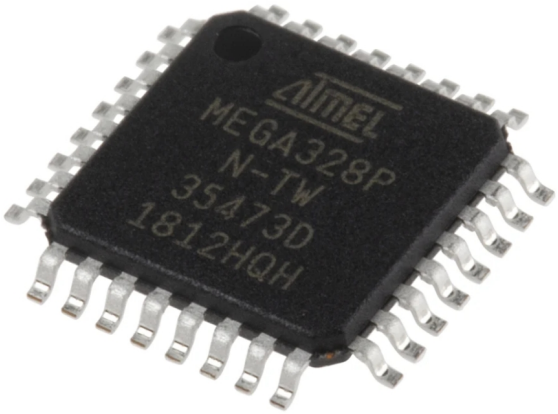
2 Wahl der Bauteile

2.1 Microcontroller

Das Herz des Projektes ist mein Microcontroller. Dieser steuert alle benötigten Komponenten an und führt nötige Berechnungen durch. Es gibt unzählige Modelle, doch in Frage kamen nur ESP32 und der Atmel Atmega328P.

Der ESP32 ist ein moderner Mikrocontroller von Espressif Systems, der besonders für IoT-Anwendungen entwickelt wurde. Er verfügt über einen leistungsstarken Dual-Core 32-Bit-Prozessor mit einer Taktfrequenz von bis zu 240 MHz. Besonders hervorzuheben ist die integrierte Unterstützung für Wi-Fi und Bluetooth, was ihn ideal für vernetzte Projekte macht. Darüber hinaus bietet der ESP32 viele Schnittstellen wie ADC, DAC, PWM, SPI, I2C und UART sowie umfangreiche Speicherressourcen mit bis zu 520 KB RAM und mehreren Megabyte Flash-Speicher. Trotz seiner Leistungsfähigkeit besitzt er verschiedene Energiesparmodi, die ihn auch für mobile oder stromsparende Anwendungen interessant machen.

Der ATmega328P ist ein weit verbreiteter 8-Bit-Mikrocontroller aus der AVR-Familie von Microchip und vor allem durch das Arduino Uno Board bekannt. Er arbeitet mit einer Taktfrequenz von bis zu 20 MHz und verfügt über 32 KB Flash-Speicher, 2 KB SRAM und 1 KB EEPROM. Er bietet grundlegende Peripherie wie analoge Eingänge, PWM, UART, SPI und I2C. Allerdings besitzt der ATmega328P keine integrierten Funkmodule. Er ist sehr energieeffizient, günstig und eignet sich besonders gut für einfache Mikrocontroller-Projekte und den Einstieg in die Embedded-Programmierung.



Auf dem Papier bietet der ESP somit deutliche Vorteile gegenüber dem Atmega328P. Ich entschied mich dennoch für den Atmel-Prozessor, da ich ihn bereits gut kannte und davon ausging, nicht viel Rechenleistung, Speicher, sowie kein Bluetooth oder W-Lan für mein Projekt zu benötigen.

2.2 GPS Modul



Um die GPS-Daten auswerten zu können, benötigte ich ein externes GPS-Modul mit einer Senderantenne. Dafür eignet sich der GY-NEO6MV2 für meinen Anwendungsbereich perfekt, da es sich um ein kompaktes und kostengünstiges Modul handelt, das auf dem bewährten u-blox NEO-6M GPS-Chip basiert. Das Modul verfügt über eine integrierte aktive Antenne, einen Spannungsregler, der eine 5V-Versorgung ermöglicht, sowie eine Pufferbatterie, die Satellitendaten speichert und einen schnelleren Fix beim Neustart erlaubt. Die Datenübertragung erfolgt über eine serielle Schnittstelle (UART) mit standardisierten NMEA-Datensätzen, die sich einfach mit dem ATMEGA auswerten lässt.

2.3 Temperatursensor

Für die Messung der Umgebungstemperatur habe ich den BME280 ausgewählt. Dieser Sensor ist kompakt, präzise und lässt sich einfach in Mikrocontroller-Projekte integrieren. In meinem Fall nutze ich nur die Temperaturfunktion – obwohl der Sensor ursprünglich auch Luftdruck messen kann. Der BME280-Sensor bietet eine hohe Temperaturgenauigkeit von ±1,0°C im Bereich von -40°C bis +85°C, was ihn ebenfalls für meine Anwendung geeignet macht.



Der BME280 arbeitet intern mit 3,3 V, ist aber auf einem Breakout-Board verbaut, das bereits einen Spannungsregler und Pegelwandler enthält. Dadurch kann er problemlos mit 5 V betrieben und auch an 5V-I2C-Bussen verwendet werden – was ihn ideal für mein Setup macht. Ich greife die Daten über I2C ab, da sich darüber mehrere Geräte einfach ansteuern lassen.

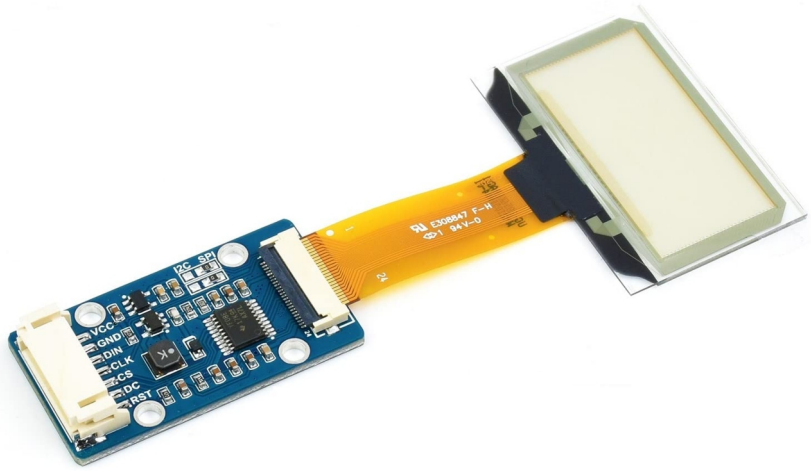
2.4 Display

Ursprünglich wollte ich die Anzeige direkt über ein Head-Up-Display (HUD) im Glas der Skibrille realisieren. Nach einiger Recherche zeigte sich jedoch schnell, dass diese Lösung in der Praxis sehr aufwändig wäre – sowohl technisch als auch optisch. Die genaue Platzierung, Lichtverhältnisse und die erforderliche Optik hätten eine deutlich komplexere Konstruktion erfordert, als im Rahmen meines Projekts umsetzbar gewesen wäre.

Stattdessen stieß ich auf eine interessante Alternative: ein transparentes OLED-Display der Firma Waveshare. Diese Art von Display ermöglicht es, Informationen direkt im Sichtfeld anzuzeigen, ohne die Durchsicht vollständig zu blockieren – genau das, was ich suchte.

Ich entschied mich für das Modell mit einer Auflösung von 128 × 64 Pixel, was für meine Zwecke – also einfache Textanzeigen wie Geschwindigkeit, Uhrzeit und Temperatur – völlig ausreichend ist. Das Display lässt sich über SPI ansteuern, was eine schnelle und stabile Datenübertragung ermöglicht. Mit einer Betriebsspannung von 5V passt es gut zum Rest meiner Elektronik.

Ein weiterer Vorteil ist die Bauform: Das Display ist extrem dünn, leicht und lässt sich gut in das Sichtfeld der Skibrille integrieren. Durch eine zusätzlich verbaute Linse wird die Darstellung optisch vergrößert und auf die Sehschärfe angepasst, sodass die Informationen auch während der Fahrt gut ablesbar bleiben.



2.5 Batterie

Um meine Schaltung zuverlässig mit Strom zu versorgen, benötigte ich einen Akku, der nicht nur ausreichend Kapazität bietet, sondern auch tiefe Temperaturen gut verkraftet. Gerade im Wintersport ist das ein kritischer Punkt, da viele Akkutechnologien bei Kälte schnell an Leistung verlieren.



Nach Abwägung verschiedener Optionen entschied ich mich für einen Lithium-Polymer-Akku (LiPo). Diese Akkus sind relativ kompakt, leicht und weit verbreitet. Ein weiterer Vorteil ist ihr flaches Design, das sich gut in das Gehäuse integrieren lässt.

Ich wählte eine Variante mit 2000 mAh, da sie einen guten Kompromiss zwischen Größe, Gewicht und Kapazität bietet. Laut Berechnungen sollte diese Kapazität ausreichen, um mein System auch bei kalten Temperaturen einen kompletten Skitag lang zu betreiben – inklusive Display, Sensoren und Mikrocontroller.

Ein Nachteil des LiPo-Akkus ist jedoch, dass die Nennspannung von nur 3,7 V, für den Betrieb meiner Schaltung nicht ausreicht.

Stromverbrauch:

Atemga	ca. 15mAh
Display	ca.35mAh
GPS Modul	ca.65 mAh
Temperatursensor	ca.2mAh
Summe	ca.117 mAh

Wegen der Kälte kann man mit einem ca. doppelten Stromverbrauch pro Stunde (234 mAh) rechnen. Also brauche ich mindestens 2000 mAh um einen Betrieb von 8h zu ermöglichen.

2.7 Spannungsversorgung

Da der LiPo-Akku nur eine Spannung von 3,7 V liefert, jedoch einige Bauteile in meinem Projekt 5 V benötigen, musste eine passende Lösung her. Dafür verwende ich den TP5400, ein praktisches Bauteil, das gleich zwei Funktionen in einem Gehäuse vereint: Er arbeitet sowohl als Step-Up-Converter als auch als Laderegler für LiPo-Akkus.

Im Aufwärtswandler-Modus hebt der TP5400 die Akkuspannung zuverlässig auf 5 V an, sodass meine gesamte Schaltung stabil betrieben werden kann. Gleichzeitig kann er über eine Micro-USB-Schnittstelle geladen werden und übernimmt dabei das komplette Lademanagement des Akkus. Das spart nicht nur Platz auf der Platine, sondern reduziert auch den Verkabelungsaufwand.

Ein weiterer Vorteil ist, dass der TP5400 relativ kompakt ist und kaum zusätzliche externe Bauteile benötigt. Er schützt den Akku vor Überladung und regelt den Ladestrom automatisch – ein wichtiges Sicherheitsfeature, besonders bei Anwendungen mit wechselnden Umgebungstemperaturen wie im Wintersport.



Durch diese Kombination aus Spannungsregelung und Ladeschaltung ist der TP5400 ideal für mein Projekt geeignet und trägt dazu bei, das System möglichst effizient und zuverlässig zu gestalten.

2.8 USB Schnittstelle

Um mein System benutzerfreundlich und flexibel zu gestalten, habe ich mich dafür entschieden, eine USB-C-Schnittstelle zu integrieren. Über diese soll zum einen der LiPo-Akku geladen, und zum anderen der Mikrocontroller programmiert werden können. So kann das gesamte System bequem über ein einziges Kabel verbunden und gewartet werden – egal ob für Software-Updates oder zum Nachladen der Batterie.





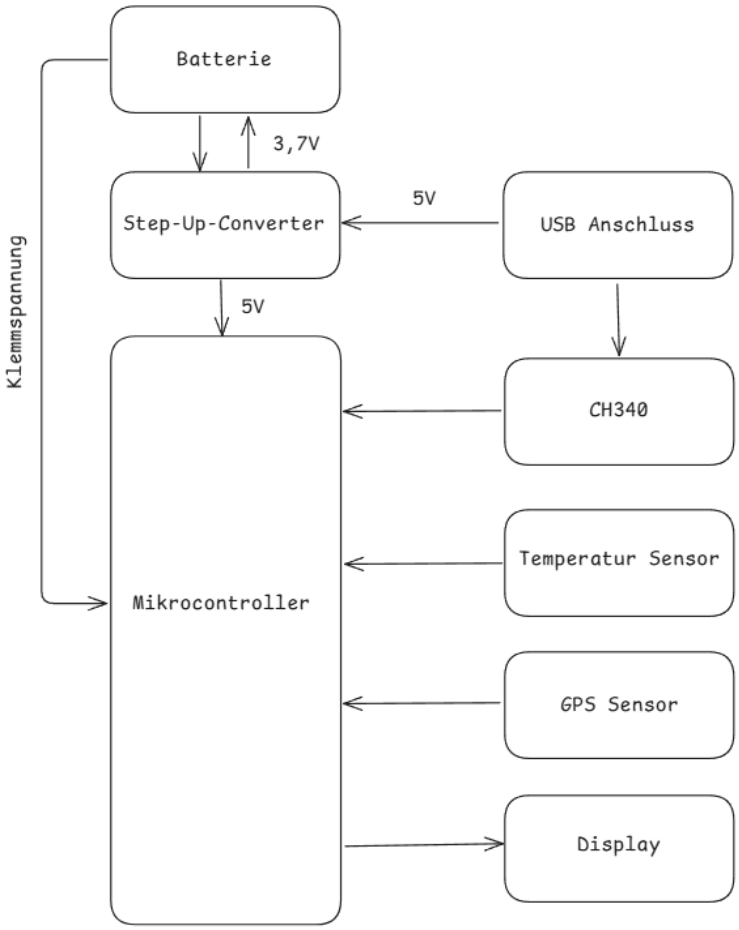
Damit die Kommunikation zwischen dem USB-Port und meinem ATmega-Mikrocontroller funktioniert, verwende ich den CH340. Dieser USB-zu-Seriell-Wandler übernimmt die Umsetzung der USB-Daten in serielle Signale, die vom Mikrocontroller verstanden werden. Der CH340 ist eine kostengünstige und weit verbreitete Lösung, die zuverlässig funktioniert und wenig Platz auf der Platine benötigt.

Die Kombination aus dem TP5400 für die Ladefunktion und dem CH340 für die Datenübertragung erlaubt mir also, Laden und Programmieren über denselben USB-C-Anschluss zu realisieren – ohne zusätzliche Anschlüsse oder Umschalter. Das vereinfacht das Design und macht das Gerät insgesamt benutzerfreundlicher und alltagstauglicher.

3 Schaltung

3.1 Blockschaltbild

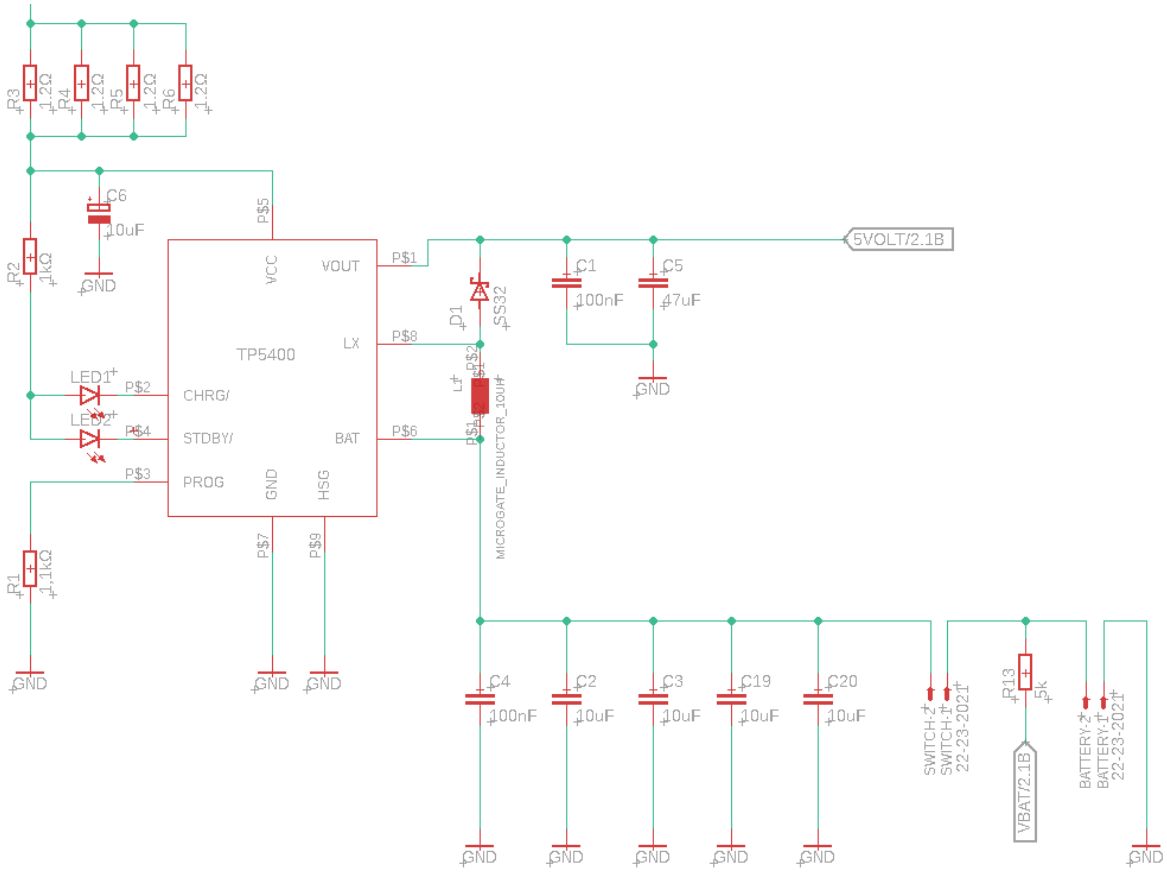
Nachdem die Bauteile festgelegt waren, begann ich mit der konkreten Planung meiner Schaltung. Um mir einen Überblick über den Aufbau zu verschaffen, erstellte ich ein Blockschaltbild, das die wichtigsten Komponenten zeigt. Der LiPo-Akku mit 3,7 V wird über einen Step-Up-Converter auf 5 V gebracht, um den Mikrocontroller und alle weiteren Komponenten zu versorgen. Über USB-C kann die Batterie geladen und der Mikrocontroller programmiert werden. Dafür sorgt der CH340, der als USB-zu-Seriell-Wandler dient. Der Mikrocontroller empfängt Daten vom BME280-Temperatursensor und vom GPS-Modul. Die gemessenen Werte werden anschließend über I2C oder SPI auf ein transparentes OLED-Display übertragen.



3.2 Spannungsversorgung/Laderegler

Die Batterie und der Schalter werden mit Pinheads mit der Platine verbunden. Stützkondensatoren sorgen für die Glättung der Ausgangsspannung (3,7V). Wie bereits erwähnt, verwende ich den TP5400, um den LiPo-Akku zu laden und gleichzeitig die 3,7 V des Akkus auf stabile 5 V zu erhöhen. Damit können alle Komponenten zuverlässig betrieben werden. Der Chip übernimmt beides: das Laden des Akkus über USB-C und die Spannungsregelung.

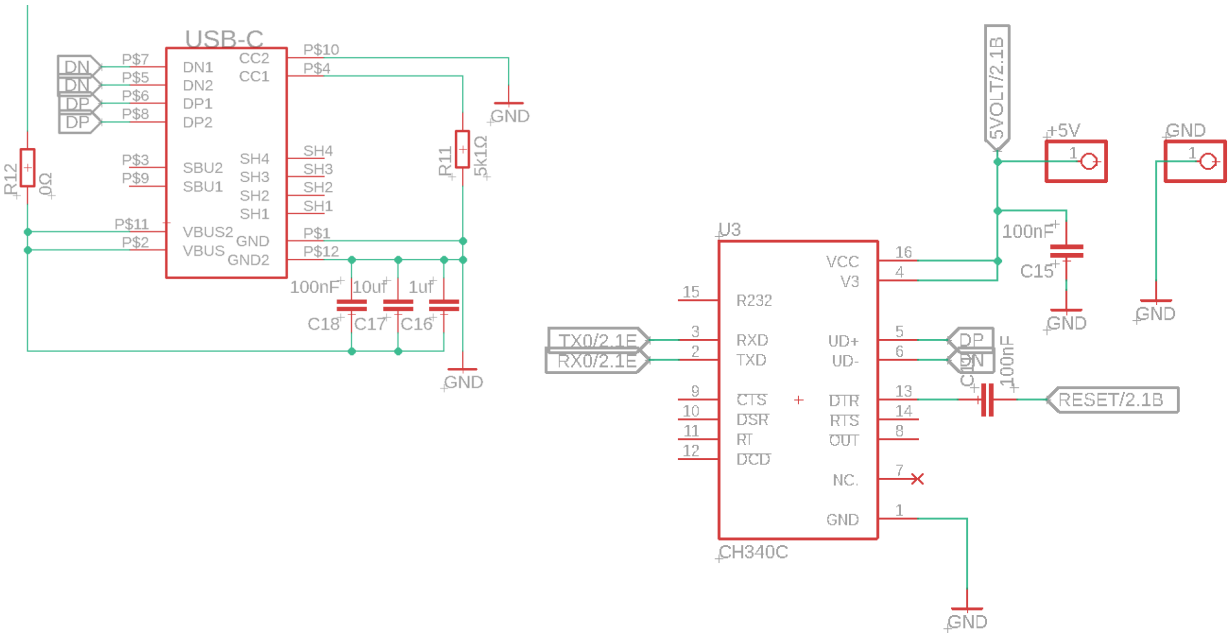
Eine Spule und eine Schottky-Diode sorgen dafür, dass die Spannung korrekt umgewandelt wird. Kondensatoren an Ein- und Ausgang glätten die Spannung und vermeiden Störungen. Zwei LEDs zeigen den Ladestatus an – ob gerade geladen wird oder der Akku voll ist.



3.3 USB Schnittstelle

Die USB-Schnittstelle meiner Schaltung wird über eine USB-C-Buchse realisiert, über die sowohl der Mikrocontroller programmiert als auch der Akku geladen werden kann. Für die Datenübertragung zwischen PC und Mikrocontroller kommt der CH340 zum Einsatz. Er dient als USB-zu-Seriell-Wandler, der die USB-Signale in ein UART-kompatibles Format übersetzt. Die beiden Datenleitungen der USB-Verbindung, D+ (DP) und D− (DN), werden direkt an den CH340 geführt. Dieser interpretiert die Signale und gibt sie über die Pins TX und RX an den Mikrocontroller weiter. Dabei war es wichtig, TX und RX korrekt mit den jeweiligen Pins des ATmega328P zu verbinden.

Für den zuverlässigen Betrieb des CH340 werden einige externe Komponenten benötigt. Ein 100nF-Kondensator zwischen VCC und GND dient zur Spannungsstabilisierung. Ein zusätzlicher Kondensator am Reset-Pin ermöglicht einen automatischen Reset des Mikrocontrollers beim Flashen über USB, wodurch der Programmierprozess deutlich komfortabler wird. Durch diese Kombination aus Hardware und Softwareintegration gelingt eine stabile, zuverlässige USB-Kommunikation zwischen Computer und Mikrocontroller.

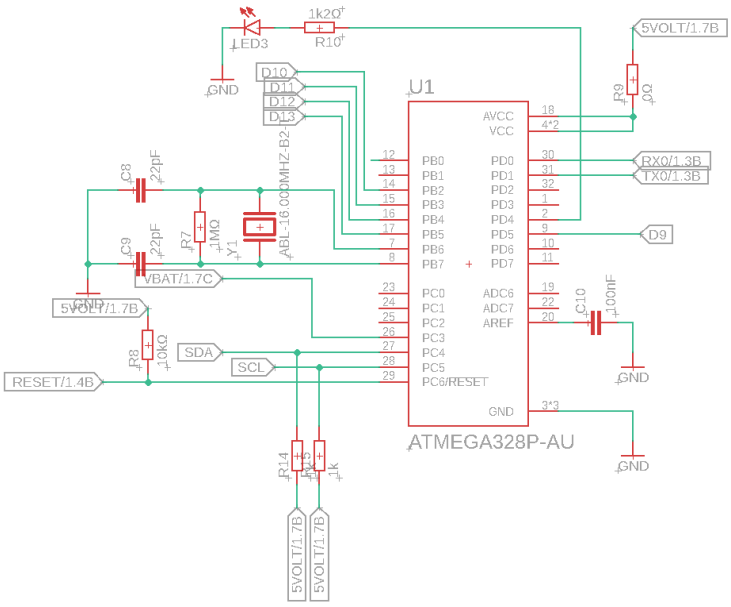


3.4 Microcontroller

Die Schaltung zeigt den grundlegenden Aufbau rund um den Mikrocontroller ATmega328P-AU, der das Herzstück meines Projekts bildet. Der Controller wird mit einer Betriebsspannung von 5 V versorgt, welche über die Pins VCC und AVCC zugeführt wird. Zur Stabilisierung dieser Versorgungsspannung ist ein 100 nF-Kondensator zwischen VCC und GND geschaltet. Der Mikrocontroller wird mit einem externen 16 MHz-Quarzoszillator betrieben, der über zwei 22 pF-Kondensatoren an GND angeschlossen ist. Diese Bauteile sorgen für einen stabilen Takt, was für eine zuverlässige Programm- und Sensorsteuerung entscheidend ist.

Zur Programmierung und Kommunikation wurde ein UART-Anschluss vorgesehen: Die Pins TXD und RXD sind über Stiftleisten zugänglich gemacht, sodass der Mikrocontroller über einen USB-zu-Seriell-Wandler programmiert oder mit dem PC kommunizieren kann. Zusätzlich sind die I2C-Pins (SDA und SCL) sowie digitale Pins (D8–D11) als Pinheads ausgeführt, um Sensoren oder ein Display einfach anschließen zu können. Eine LED ist an Pin PD4 (Digitalpin 4) angeschlossen und dient zur Funktionskontrolle der Schaltung – sie zeigt zum Beispiel an, ob das System aktiv ist.

Auch ein Reset-Taster ist eingebaut: Er verbindet den Reset-Pin über einen Pull-Up-Widerstand mit VCC und kann das System per Tastendruck zurücksetzen. Die Versorgungsspannung kann zwischen Batterie (VBAT) und USB (5V) umgeschaltet werden – dies erfolgt über die separate Spannungsleitung und Dioden in der restlichen Schaltung, die in diesem Ausschnitt nicht zu sehen sind.

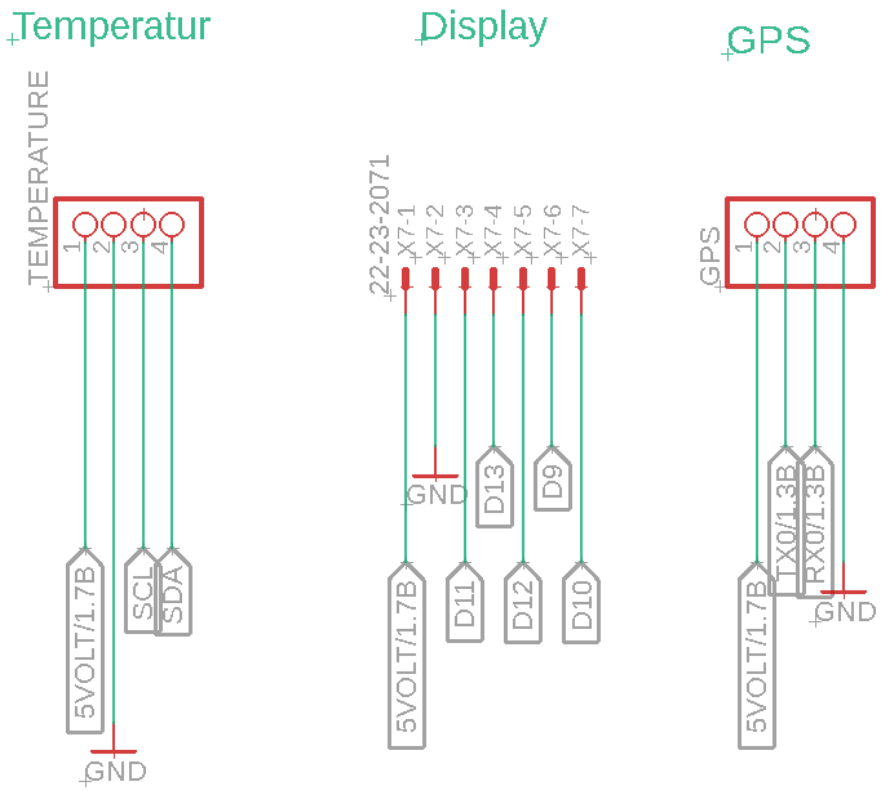


3.5 Sensorik

Zur Erfassung der Umgebungsdaten kommen ein GPS-Modul und ein BME280-Temperatursensor zum Einsatz. Das GPS-Modul ist über eine serielle Verbindung (TX/RX) mit dem ATmega328P verbunden und über Pinheads auf der Platine befestigt, was einen sicheren Kontakt und einfachen Austausch ermöglicht. Es liefert kontinuierlich Positionsdaten zur Bestimmung von Geschwindigkeit und Uhrzeit.

Der BME280-Sensor misst die Temperatur über die I2C-Schnittstelle, die dank geringer Verdrahtung zuverlässig arbeitet. Auch dieser Sensor ist über Pinheads angebunden. Für eine korrekte Funktion des I2C-Busses sind Pull-up-Widerstände an SDA und SCL notwendig – in meinem Aufbau 4,7 kΩ –, um stabile High-Pegel zu gewährleisten.

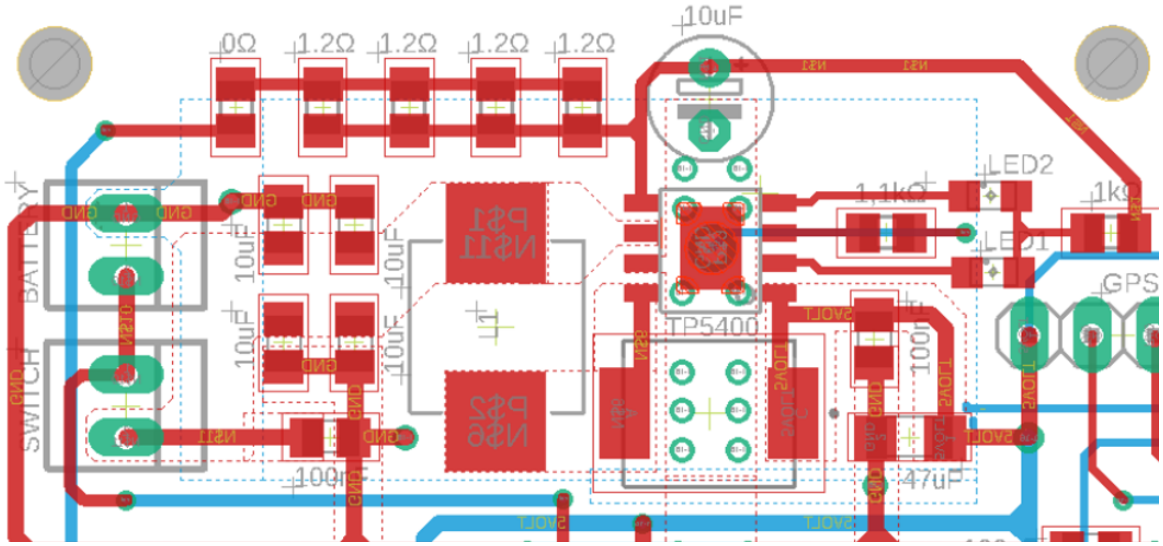
Zur Ausgabe der Daten dient ein Display, das über SPI kommuniziert und mittels abgewinkelter (90°) Pinheads montiert ist.



4 Platine

4.1 Spannungsversorgung

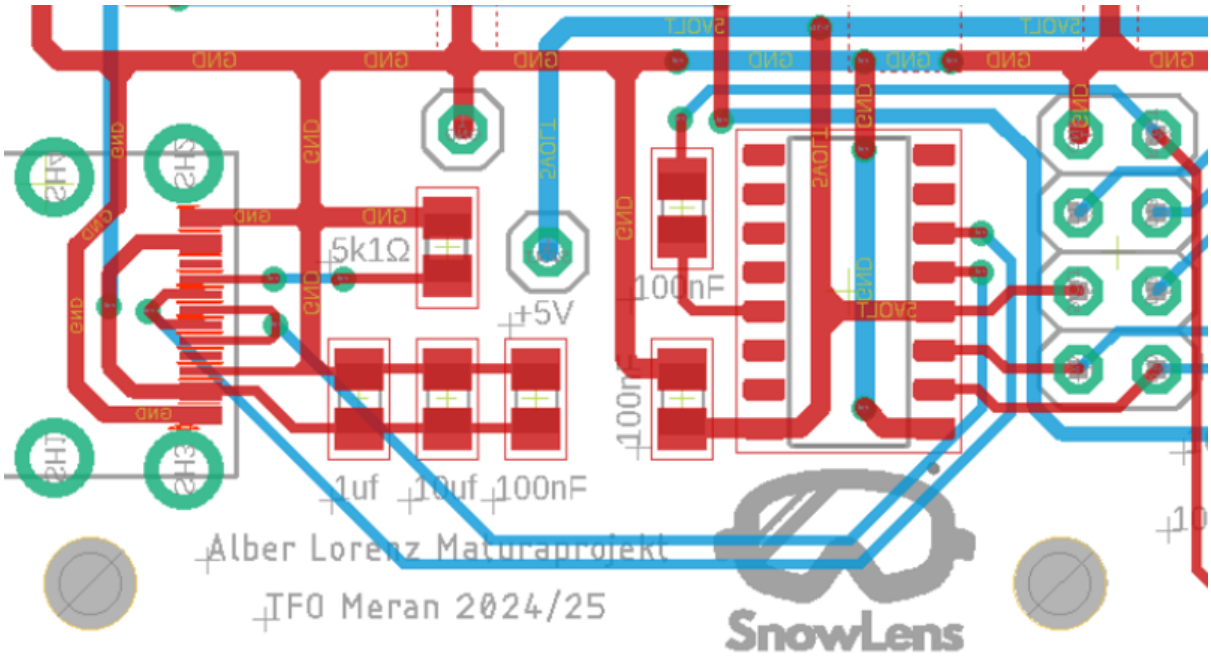
Die Bauteile auf der Spannungsversorgungsplatine wurden möglichst übersichtlich und platzsparend angeordnet. Ganz links befinden sich die Pads für den Anschluss der Batterie sowie der Ein-/Aus-Schalter mit dem die gesamte Schaltung ein- oder ausgeschaltet werden kann. Direkt daneben sieht man mehrere Widerstände und Kondensatoren, die gemeinsam mit der Spule und dem TP5400 die Stromversorgung aufbauen. Der Spannungsregler sorgt dafür, dass aus der Batteriespannung eine stabile 5V Spannung erzeugt wird. Rechts daneben sind weitere Kondensatoren platziert, die helfen, die erzeugte Spannung zu glätten und störungsfrei weiterzugeben. Auf der Unterseite der Platine befindet sich ein durchgehendes Massepolygon, das als gemeinsame Rückleitung für alle Schaltungsteile dient und Störungen reduziert. Auf der Oberseite ist ein separates Polygon für die 5 V Versorgung angelegt, über das die stabilisierte Spannung effizient zu den einzelnen Komponenten verteilt wird.



4.2 USB Schnittstelle

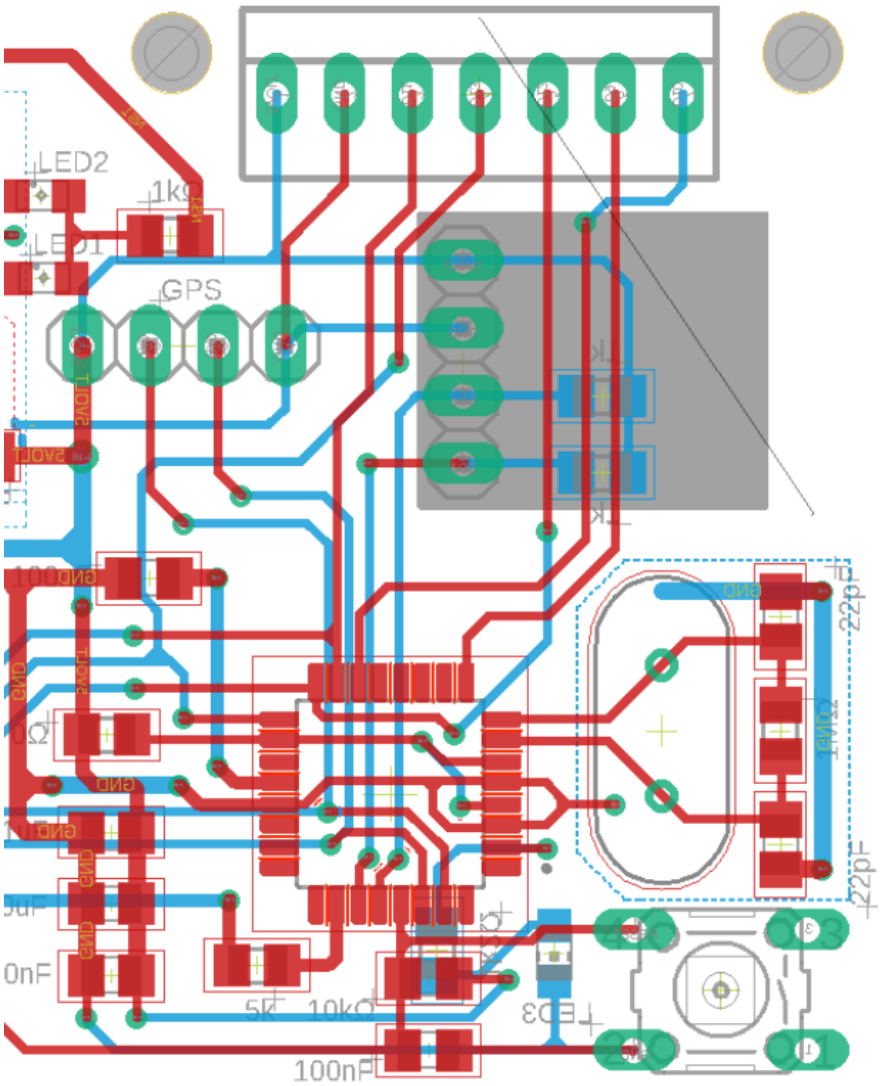
Die USB-Schnittstelle zur Programmierung und Datenübertragung befindet sich unten links auf der Platine. Sie besteht aus einer USB-C-Buchse, über die sowohl Strom eingespeist als auch eine serielle Verbindung zum Mikrocontroller hergestellt werden kann. Direkt daneben ist der CH340-IC positioniert, der als USB-zu-Seriell-Wandler dient. Dieser Chip übernimmt die Kommunikation zwischen dem Computer und dem Mikrocontroller, indem er die USB-Signale in serielle UART-Signale umwandelt.

Um die Funktion des CH340 zu unterstützen, wurden einige SMD-Bauteile wie Pull-Up-Widerstände, ein 5,1 kΩ-Widerstand für die USB-C-Erkennung sowie Abblockkondensatoren in unmittelbarer Nähe platziert. Auch die USB-Datenleitungen D+ und D– verlaufen möglichst kurz und symmetrisch vom Anschluss zum CH340, um Signalstörungen zu minimieren.
Die 5 V, die über USB eingespeist werden, stehen über eine Diode und einen nachgeschalteten Spannungsregler auch der restlichen Schaltung zur Verfügung, wenn kein Akku angeschlossen ist. Damit kann die gesamte Platine auch direkt über USB betrieben oder programmiert werden, was besonders bei der Entwicklung und beim Testen des Projekts hilfreich war.



4.3 Microcontroller & Sensorik

Der Mikrocontroller auf der rechten unteren Seite der Platine platziert. Direkt daneben befindet sich der 16 MHz Quarz, der zusammen mit zwei 22pF-Kondensatoren den Takt für den Mikrocontroller vorgibt. Dabei war es besonders wichtig, den Quarz so nah wie möglich an den entsprechenden Pins des Controllers zu platzieren, um Störungen und Taktinstabilitäten durch zu lange Leiterbahnen zu vermeiden. Beim Platzieren der Stützkondensatoren achtete ich darauf, sie so nahe wie möglich am Controller zu platzieren, damit sie die Eingangsspannung komplett glätten. Der Taster und die Pinheads der Sensoren und Aktoren hingegen wurden nun so platziert, dass sie später im Gehäuse leicht zugänglich waren.



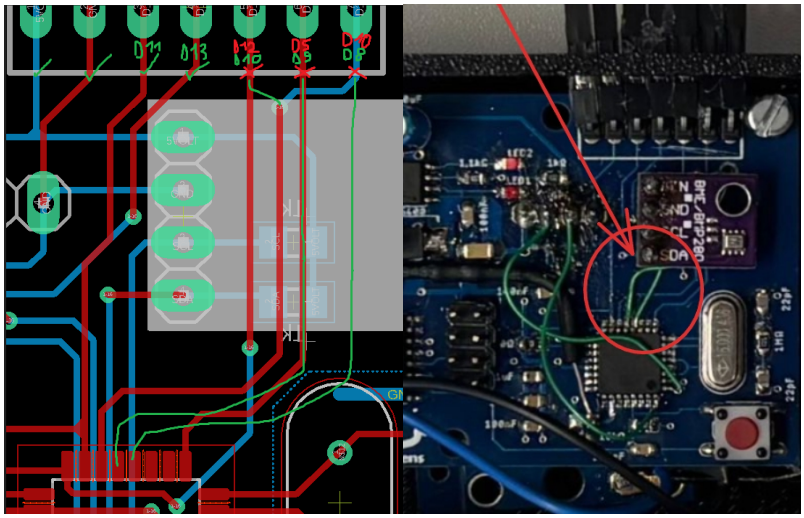
4.4 Vorgenommene Änderungen

Beim Zeichnen und Umsetzen der Schaltung unterliefen mir leider mehrere Fehler, die sich im späteren Projektverlauf als besonders zeitaufwendig herausstellten. Einer der Hauptfehler war die fehlerhafte Verdrahtung einiger Pins des OLED-Displays: Ich verband bestimmte Steuerleitungen mit den falschen Pins des Mikrocontrollers, wodurch das Display zunächst keinerlei Funktion zeigte. Zusätzlich vertauschte ich die RX- und TX-Leitungen des GPS-Moduls, was dazu führte, dass keine Daten empfangen oder gesendet wurden. Diese Art von Fehlern sind auf den ersten Blick schwer zu erkennen und äußerten sich erst im laufenden Betrieb durch fehlende oder fehlerhafte Ausgaben auf dem Display.

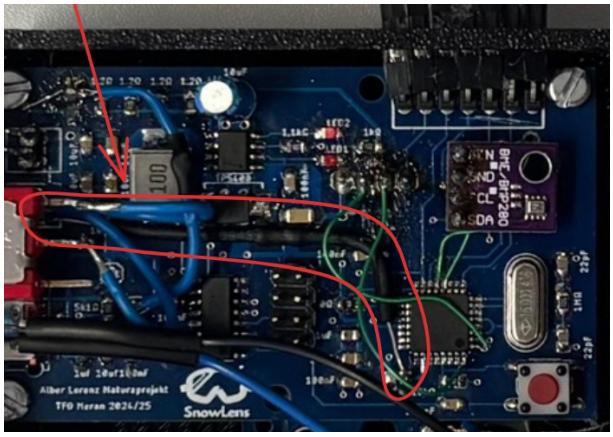
Ein weiterer Fehler betraf die Spannungsversorgung der Platine. Beim Entwurf vergaß ich, eine Diode zwischen der Batterieversorgung und dem Spannungsregler zu integrieren. Dadurch wurde der Mikrocontroller beim Betrieb über USB-C nicht mit den erforderlichen 5 Volt versorgt, sondern lediglich mit der Batteriespannung von 3,7 Volt. Das führte dazu, dass einzelne Komponenten nicht ordnungsgemäß funktionierten oder sich sogar komplett abschalteten.

Diese Fehler wurden erst nach intensiven Tests sichtbar und erforderten viel Zeit und Geduld, um sie systematisch zu identifizieren und zu korrigieren. Besonders aufwändig war die nachträgliche Korrektur der Displayanschlüsse. Um die fehlerhaften Verbindungen zu beheben, musste ich gezielt Leiterbahnen auf der Platine aufritzen. Anschließend bohrte ich mithilfe von Herrn Professor De Tomaso ein Loch durch die Platine, um mit kurzen Kabeln eine direkte Verbindung zwischen den korrekten Pins des Mikrocontrollers auf der Oberseite und den THT-Anschlüssen des OLED-Displays auf der Unterseite herzustellen. Diese Anpassungen waren zwar mühsam, ermöglichten aber letztendlich einen funktionierenden Aufbau, ohne dass eine neue Platine gefertigt werden musste.

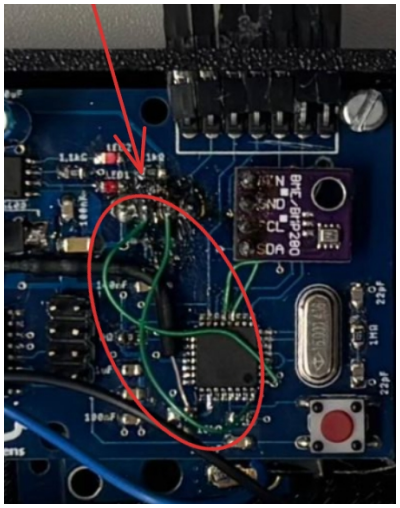
Korrekturen am Display:



Korrektur der Betriebsspannung:



Korrektur von TX und RX:



5 Software

Ein zentraler Bestandteil meines Projekts ist die Software, die sämtliche Sensordaten erfasst, verarbeitet und in Echtzeit auf dem Display darstellt. Sie verbindet die verschiedenen Hardware-Komponenten miteinander und sorgt dafür, dass Geschwindigkeit, Uhrzeit, Temperatur und Akkustand angezeigt werden. In diesem Kapitel beschreibe ich den Aufbau, die Funktionsweise und die wichtigsten Elemente des Programmcodes.

5.1 Aufbau

Einbinden der Bibliotheken
Am Anfang werden alle benötigten Bibliotheken eingebunden, um GPS, Temperaturmessung, Displaysteuerung und serielle Kommunikation zu ermöglichen.

Definition der Pins und Variablen
Es werden alle verwendeten Pins (für OLED, GPS, LED, Batterieüberwachung) sowie globale Variablen definiert, um später auf sie zugreifen zu können.

Initialisierung im Setup
Im Setup-Teil werden die Module wie das Display, der Temperatursensor und das GPS-Modul gestartet. Zudem erfolgt eine erste Anzeige zur Systemprüfung auf dem Display.

Hauptschleife
In der Hauptschleife wird alle 500 ms geprüft, ob neue GPS-Daten verfügbar sind. Gleichzeitig wird die Batteriespannung ausgelesen und in Prozent umgerechnet. Wenn der Temperatursensor verfügbar ist, wird auch die Temperatur gemessen und angepasst.

Anzeige der Daten
Alle erfassten Informationen (Uhrzeit, Geschwindigkeit, Temperatur und Batteriestand) werden auf dem OLED-Display übersichtlich dargestellt. Das Display ist dabei um 270° gedreht, da es vertikal im Gehäuse verbaut ist und somit lesbar ist.

5.2 GPS Modul

Am Anfang wird das GPS-Modul über die Software-Serial-Schnittstelle an den digitalen Pins 2 (RX) und 3 (TX) eingebunden und mit einer Baudrate von 9600 initialisiert. In der Hauptschleife wird laufend geprüft, ob neue GPS-Daten eintreffen. Die empfangenen NMEA-Daten werden mithilfe der TinyGPSPlus-Bibliothek decodiert. Dabei interessieren vor allem zwei Werte:

Zeit: Die Uhrzeit wird über gps.time ausgelesen. Da GPS-Zeit standardmäßig in UTC geliefert wird, wird für die mitteleuropäische Zeitzone ein Offset von +2 Stunden hinzugefügt. Das Format der Anzeige ist hh:mm:ss.

Geschwindigkeit: Die Geschwindigkeit wird mit gps.speed.kmph() in Kilometern pro Stunde ausgelesen und auf eine Nachkommastelle formatiert. Wird kein gültiges Signal empfangen, erscheint auf dem Display „N.D.“ (nicht definiert).
Das GPS-Modul ist somit zentral für die dynamische Anzeige während der Fahrt verantwortlich.

So wird das GPS Modul ausgelesen:

```
for (unsigned long start = millis(); millis() - start < 500;)
{
    while (gpsSerial.available())
    {
        char c = gpsSerial.read();
        if (gps.encode(c))
            newData = true;
    }
}
```

5.3 Temperatursensor

Der BMP280 ist ein I2C-basierter Sensor, der an den Standardpins (A4 und A5 beim ATmega328P) betrieben wird. Der Sensor wird beim Start des Programms mit tempSensor.begin() aktiviert. Falls er erkannt wird (Adresse 0x76 oder 0x77), wird dies am Display signalisiert („Temp sensor online“).
In der Hauptschleife wird regelmäßig die aktuelle Temperatur in Grad Celsius über tempSensor.readTemperature() ermittelt. Da der Sensor nahe an der Elektronik sitzt, die sich leicht erwärmt, wird ein Offset von -8 °C angewendet, um die Messung realistischer zu gestalten. Der Offset-Wert wurde anhand von entsprechenden Messungen ermittelt.

Die Temperatur kann dank der Library sehr einfach ausgelesen werden:

```
float temperature = tempSensor.readTemperature();
```

Für die gesamte Software siehe in den Anhängen.

5.4 Display

Der transparente OLED-Bildschirm wird über das SPI-Protokoll angesteuert. Die Signalleitungen CS, DC und RESET sind dabei mit den Pins 10, 9 und 8 des Mikrocontrollers verbunden. Zur Ansteuerung verwende ich die U8g2-Bibliothek, die eine einfache Darstellung von Texten und Symbolen auf dem Display ermöglicht. Da der OLED aufgrund der Einbausituation um 90° gedreht in der Skibrille verbaut ist, wird das Bild softwareseitig mit der Einstellung U8G2_R3 um 270° rotiert.

Das Display wird im sogenannten Page-Buffer-Modus betrieben. Dabei wird der Bildinhalt zunächst vollständig im Speicher des Mikrocontrollers aufgebaut und anschließend zeilenweise mit den Befehlen firstPage() und nextPage() auf das Display übertragen. Dadurch bleibt die Anzeige flimmerfrei und sauber aufgebaut. Auf dem OLED werden alle gesammelten Daten wie Geschwindigkeit, Uhrzeit, Temperatur und Akkustand übersichtlich und kompakt dargestellt.

Ich entscheid mich für folgendes Layout auf dem Display:



Eigentlich war es meine Aufgabe, die Display-Auswertung ohne das Verwenden einer Library zu realisieren, um den Mikrocontroller nicht zu sehr zu beanspruchen und Speicher zu sparen. Ich schaffte dies jedoch leider nicht und griff somit auf eine Bibliothek zurück. Das ganze Programm inklusive Bibliothek verwendet 98% des Speichers und ist somit im kritischen Bereich.

So sieht ein Teil der Software zur Temperaturanzeige auf dem Display aus:

```
display.firstPage();
do {
  display.setFont(u8g2_font_6x10_tf);

  // Uhrzeit anzeigen, wenn GPS gültig
  if (gps.time.isValid()) {
    char timeStr[16];
    int hour = (gps.time.hour() + 2) % 24; // Zeitzone Italien
    sprintf(timeStr, "%02d:%02d:%02d", hour, gps.time.minute(),
gps.time.second());
    display.drawStr(0, 22, timeStr);
  }
```

Für die gesamte Software siehe in den Anhängen.

5.5 Batteriestand

Zur Überwachung des Akkuzustands wird die Batteriespannung am analogen Eingang A3 eingelesen. Dabei wird die Spannung zunächst mithilfe des analogRead()-Befehls gemessen und anschließend in eine reale Spannung umgerechnet. Da die Referenzspannung des Mikrocontrollers 5 V beträgt, ergibt sich die tatsächliche Batteriespannung durch die Formel:

```
float batteryVoltage = (analogRead(BATTERY_PIN) / 1023.0) * 5.0;
uint8_t batteryPercentage = getBatteryPercentage(batteryVoltage);
```

Diese Spannung wird anschließend durch eine Funktion in eine Prozentangabe umgerechnet, wobei typische Spannungswerte eines Lithium-Ionen-Akkus berücksichtigt werden. So wird beispielsweise eine Spannung von 4,2 V als 100 % und 3,3 V als etwa 10 % gewertet. Der berechnete Prozentsatz wird auf dem OLED-Display dargestellt, sodass jederzeit ein grober Überblick über den aktuellen Ladezustand der Batterie möglich ist.

```
uint8_t getBatteryPercentage(float voltage)
{
    if (voltage >= 4.2) return 100;
    if (voltage >= 4.1) return 90;
    if (voltage >= 4.0) return 80;
    if (voltage >= 3.9) return 70;
    if (voltage >= 3.8) return 60;
    if (voltage >= 3.7) return 50;
    if (voltage >= 3.6) return 40;
    if (voltage >= 3.5) return 30;
    if (voltage >= 3.4) return 20;
    if (voltage >= 3.3) return 10;
    if (voltage >= 3.25) return 5;
    return 0;
}
```

Für die gesamte Software siehe in den Anhängen.

6 Gehäuse

6.1 Voraussetzungen

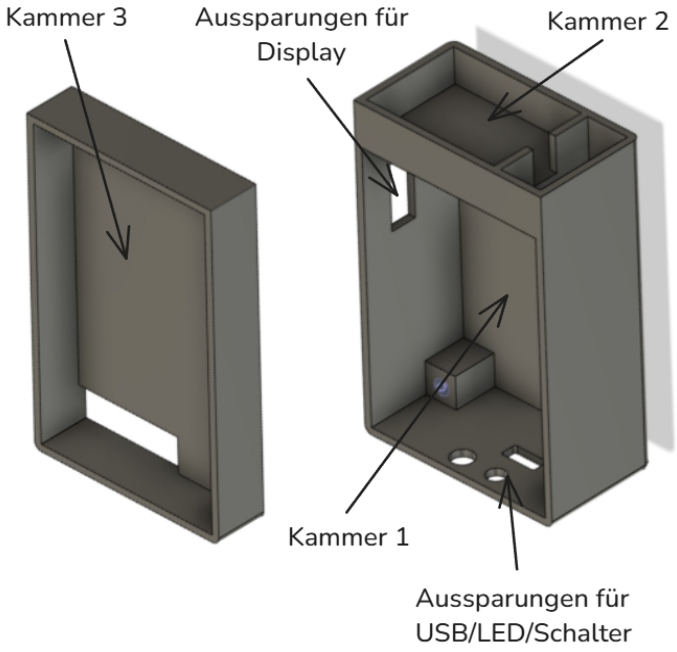
Bevor ich mit dem Bau des Gehäuses begann, überlegte ich mir genau, welche Anforderungen es erfüllen muss. Die Box soll die gesamte Elektronik - also die Platine mit den darauf montierten Sensoren sowie die Stromversorgung - zuverlässig vor Feuchtigkeit und mechanischen Einflüssen schützen. Da das System beim Skifahren zum Einsatz kommt, sind ein kompakter Aufbau und eine wetterfeste Ausführung besonders wichtig.

Ich plante daher ein passgenaues Gehäuse, das genügend Platz für alle Bauteile bietet. Schon beim Entwurf berücksichtigte ich gezielt Öffnungen für den Taster, die Status-LED, den USB-C-Anschluss sowie für das Kabel zum Display in der Skibrille.

6.2 Design & Umsetzung

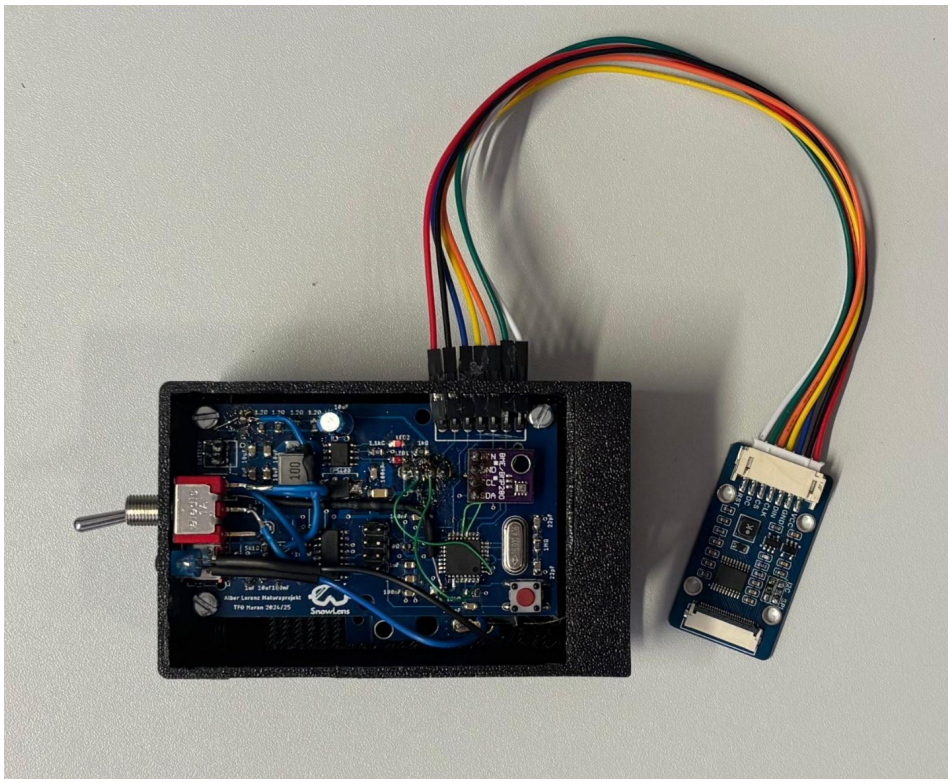
Um dieses Konzept nun umzusetzen, informierte ich mich über Fertigungsmethoden, um so eine Box herzustellen. Ich kam zum Schluss, dass sich ein 3D-Druck gut wegen seiner relativ simplen Herstellungsweise eignet. Da das 3D-Druck Filament jedoch nicht wasserabweisend ist, muss ich die Öffnungen und Ritzen nach dem Einbau der Elektronik mit Heiskleber füllen und danach mit Haarspray versiegeln.

Ich plante mein Gehäuse in drei getrennte Kammern aufzuteilen, um die einzelnen Komponenten funktional und sicher unterzubringen. Die erste Kammer nimmt die Hauptplatine samt Elektronik auf. In der zweiten Kammer wird die GPS-Antenne platziert, so ausgerichtet, dass sie stets nach oben zeigt und dadurch optimal Satellitensignale empfangen kann. Die dritte Kammer ist für die Batterie vorgesehen. An der Rückseite integrierte ich auch noch einen einfachen Befestigungsmechanismus, welcher es ermöglicht, die Box mithilfe von Gummibändern am Band der Skibrille zu befestigen. Mit diesem Plan begann ich also mit dem Zeichnen eines 3D-Modells in der Fusion 360 und kam zu folgendem Ergebnis:



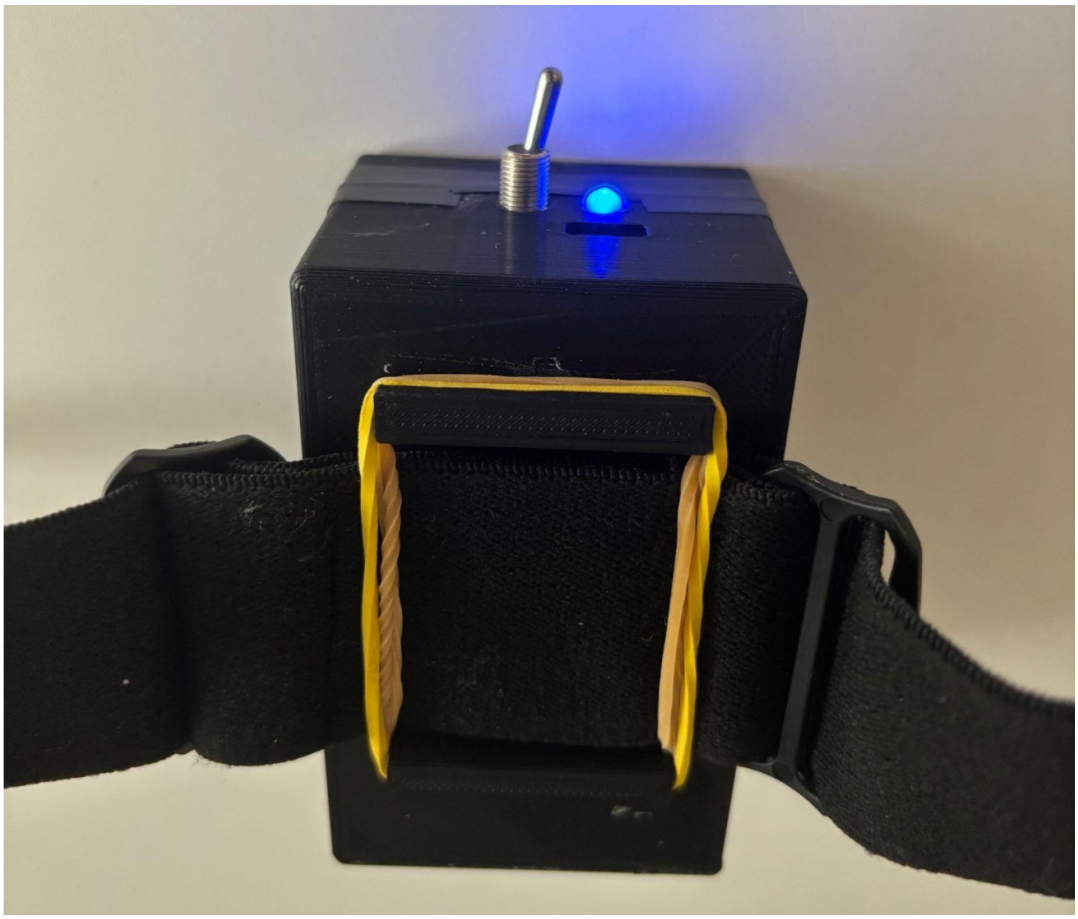
Mein Klassenkamerad Felix Steck hat einen 3D-Drucker zu Hause und druckte mir freundlicherweise die beiden Elemente. Die beiden Bauteile werden nach der Montage der Platine miteinander verklebt. Natürlich wird auch noch ein Deckel über Kammer 2 und 3 angebracht, nachdem die Batterie und der Empfänger eingesetzt wurden.

6.3 Ergebnis:





Halterung an Brille:



6.4 Einbau in Skibrille

Um nun mein Projekt zu vollenden, kaufte ich mir eine handelsübliche Skibrille. Sie besitzt speziell für Demonstrationszwecke ein transparentes Glas, sodass die verbaute Technik und insbesondere das OLED-Display auch von außen gut sichtbar sind. Wie im vorherigen Kapitel beschrieben, befestigte ich die Elektronikbox mit dem zuvor integrierten Befestigungsmechanismus an der Rückseite des Gummibands der Skibrille. So konnte die Box stabil montiert und gleichzeitig unauffällig platziert werden.

Anschließend positionierte ich das OLED-Display auf der Innenseite der Skibrille, genau an der Stelle, an der es später für den Träger sichtbar sein sollte. Um das Flachbandkabel vom Display zur Box führen zu können, schnitt ich vorsichtig eine Öffnung in den seitlichen Rahmen der Skibrille. Durch diese Durchführung zog ich das Flachbandkabel und richtete es so aus, dass es nicht im Sichtfeld lag und gleichzeitig genug Spielraum für Bewegungen bot.

Nachdem das Display an der vorgesehenen Position lag, fixierte ich es mit einem stabilen Klebstoff direkt auf dem Brillenglas. Das Flachbandkabel sicherte ich zusätzlich mit Heißkleber entlang des Brillenrands, um es vor mechanischer Belastung und Witterungseinflüssen zu schützen. Das andere Ende des Flachbandkabels, das seitlich aus der Brille herausragte, verband ich anschließend mit der Displayplatine, die wiederum über die entsprechenden Pinheads mit der Hauptplatine in der Box kommuniziert.

Um die Displayplatine sicher zu montieren, befestigte ich sie mit zwei kleinen Kabelbindern direkt am Gummiband der Skibrille. Diese Lösung war einfach, leicht und zugleich stabil genug, um die Platine in Position zu halten, ohne die Bewegungsfreiheit oder den Tragekomfort zu beeinträchtigen.

Montage des Flabandkabels:



Gesamtes Ergebnis:



6.5 Optik

Da mir von Anfang an bewusst war, dass ein Display in sehr geringer Entfernung zum Auge schwer lesbar sein würde, plante ich bereits früh eine optische Unterstützung ein. Das menschliche Auge ist nicht in der Lage, auf sehr kurze Distanzen scharf zu stellen - insbesondere dann nicht, wenn das Objekt, wie in meinem Fall das OLED-Display, nur wenige Zentimeter vom Auge entfernt ist. Dadurch erschienen die angezeigten Informationen unscharf und waren beim Tragen der Skibrille kaum erkennbar.

Um dieses Problem zu lösen, entschied ich mich, eine handelsübliche Handlupe mit einer dreifachen Vergrößerung zu verwenden. Ich montierte das Lupenglas in einem Abstand von etwa 2 cm vor dem Display in der Skibrille. Durch diese optische Vergrößerung wurden die dargestellten Inhalte für das Auge deutlich lesbarer. So konnte ich die aktuelle Geschwindigkeit und andere Informationen auch während der Fahrt problemlos erkennen.



7 Projektmanagement

7.1 Wochenberichte

Im Verlauf des Projekts war es verpflichtend, zu jeder einzelnen Projektwoche einen schriftlichen Wochenbericht zu verfassen. Sie halfen dabei, den eigenen Arbeitsfortschritt regelmäßig zu reflektieren, mögliche Verzögerungen frühzeitig zu erkennen und gleichzeitig eine lückenlose Projektdokumentation zu gewährleisten.

Der Bericht bestand aus mehreren standardisierten Abschnitten. Im ersten Teil wurde in einem kurzen Fließtext beschrieben, welche Tätigkeiten während der Woche durchgeführt wurden, z.B. der Aufbau von Schaltungen, das Testen von Funktionen oder die Arbeit am Gehäuse. Auch Herausforderungen oder Probleme wurden hier erwähnt, ebenso wie deren Lösung.

Anschließend wurde der Projektverlauf mit dem ursprünglich erstellten GANTT-Diagramm verglichen, um festzustellen, ob man sich noch im Zeitplan befand. Bei Abweichungen mussten entsprechende Maßnahmen formuliert werden, um wieder in den Plan zurückzufinden.

Im unteren Bereich des Wochenberichts wurde der Zeitaufwand dokumentiert. Dazu zählte, wie viele Stunden im Labor an der Schule gearbeitet wurden und ob zusätzliche Zeit im offenen Labor oder zu Hause aufgewendet wurde. Letzteres war bei mir regelmäßig der Fall, da ich meine Wochenziele nicht immer während der regulären Schulzeit erreichen konnte und daher abends oder am Wochenende weiterarbeitete.

So entstand über die gesamte Projektdauer hinweg ein durchgehendes Protokoll aller geleisteten Arbeiten.

Ein Beispiel finden Sie in den Anhängen.

7.2 Beraterstunden

Während des Projekts war es erforderlich, jede Form von Unterstützung durch Mitschüler oder Lehrkräfte zu dokumentieren. Dazu wurde eine spezielle Tabelle geführt, in der sogenannte Beraterstunden eingetragen wurden. Diese Tabelle diente der Nachvollziehbarkeit externer Hilfeleistungen und zeigte auf, wann und in welchem Umfang Unterstützung in Anspruch genommen wurde.

Für jeden Eintrag wurden das Datum, die Dauer der Beratung in Stunden sowie der Name der beratenden Person festgehalten. Dabei spielte es keine Rolle, ob die Hilfe technischer Natur war, sich auf Softwarefragen bezog oder sich beispielsweise auf die Planung des Gehäuses konzentrierte – jede Unterstützung wurde dokumentiert.

Projekt: SnowLens
Alber Lorenz - 4B EL
Auflistung Stunden externer Berater
Seite: 1

Datum	Anzahl Stunden	Name Berater
15.10.2024	0,5	Steck Felix
19.11.2024	0,25	Dominik Gamper
3.12.2024	0,25	Liam Bakay Donati
28.01.2025	0,25	Prof. De Tomaso
11.03.2025	1	Prof. De Tomaso
24.03.2025	0,5	Prof. De Tomaso
27.03.2025	0,25	Steck Felix
1.04.2025	0,25	Preims Florian

Anmerkung: jede angebrochene Viertelstunde ist einzutragen, also wird im Feld "Anzahl Stunden" entweder "0,25" stehen (wenn weniger als 15 Minuten, z.B. 10 Minuten), "0,5" bei einer Zeit zwischen 16 Minuten und 30 Minuten usw. Zeiten unter 5 min. brauchen nicht protokolliert werden.

7.3 Kostenberechnung

Stückzahl	Bauteil	Modell	Bezugsquelle	Stückpreis	Gesamtpreis	1000er Serie
Sensoren/Aktoren						
1	GPS Modul	GY-NEO6MV2	Aliexpress	2,59 €	2,59 €	2.331,00 €
3	Temperatursenso	BMP 280	Aliexpress	1,07 €	3,21 €	963,00 €
1	Display	Waveshare 1.51 Transparent OLED	Aliexpress	21,40 €	21,40 €	19.260,00 €
ICs						
2	Mycrocontroller	Microchip Tech ATMEGA328P-AU	LCSC	2,23 €	4,46 €	1.526,20 €
2	Stepup Converter	TOPPOWER TP5400	LCSC	1,11 €	2,22 €	130,10 €
2	USB-Seriell-Wand	WCH CH340G	LCSC	0,51 €	1,02 €	294,70 €
Zusätzlcihe Teile						
2	USB-Port	BXCONN UC09-0O16F00P11	LCSC	0,68 €	1,36 €	50,40 €
1	Quarz x 5	TAITIEN Elec XIHCELNANF-16MHZ	LCSC	0,53 €	0,53 €	49,20 €
1	Schalter	1MS4T1B1M1RE	Farnell	2,22 €	2,22 €	1.430,00 €
1	Taster	TS02-66-50-BK-160-LCR-D	DigiKey	0,11 €	0,11 €	57,00 €
1	Pinheads x 20	XFCN PZ254V-11-04P	LCSC	0,40 €	0,40 €	24,80 €
1	LED x 10	LSM0805412V	DigiKey	3,05€	3,05 €	135,78 €
Widerstände						
1	0Ω x 100	UNI-ROYAL 0805W8F0000T5E	LCSC	0,17 €	0,17 €	1,30 €
1	1,2Ω x 100	UNI-ROYAL 0805W8F120KT5E	LCSC	0,33	0,33	2,60 €
1	1kΩ x 100	UNI-ROYAL 0805W8F1001T5E	LCSC	0,17	0,17 €	1,40 €
1	4,7kΩ x 100	FOJAN FRC0805J472 TS	LCSC	0,13 €	0,13 €	1,00 €
1	10kΩ x 100	FOJAN FRC0805J103TS	LCSC	0,12 €	0,12 €	0,90 €
1	1MΩ x 100	FOJAN FRC0805J105 TS	LCSC	0,14 €	0,14 €	1,10 €
Kondensatoren						
1	22pF x 100	CCTC TCC0805COG220J500BT	LCSC	0,41 €	0,41 €	3,10 €
1	100nF x 100	YAGEO CC0805KRX7R9BB104	LCSC	0,44 €	0,44 €	3,50 €
1	1uF x 100	CL21B105KBFNNNE	LCSC	0,47 €	0,47 €	4,67 €
1	10uF x 100	CL21A106KAYNNNE	LCSC	0,21 €	0,21 €	4,32 €
1	10uF x 100	CL21A476MQYNNNE	LCSC	0,30 €	0,30 €	4,73 €
1	10uF ElKo	CX KS106M025C07RR0VH2FP0	LCSC	0,62 €	0,62 €	15,90 €
Summe				39,41 €	46,08 €	26.296,70 €

Für mein Projekt habe ich eine detaillierte Kostenaufstellung aller benötigten Bauteile erstellt. Die Gesamtsumme der einzelnen elektronischen Komponenten beträgt 46,08 €. Dabei wurden jedoch einige Bauteile, wie z. B. Widerstände oder Standardverbinder von der Schule zur Verfügung gestellt und mussten nicht zusätzlich angeschafft werden.

Zusätzlich zu den elektronischen Teilen fielen noch kleinere Ausgaben für Materialien wie Klebestreifen, Kabel oder Kondensatoren an. Diese belaufen sich schätzungsweise auf 15€. Das Gehäuse, welches ein Freund für mich druckte, kostete 10 €. Ich verlor außerdem den Überblick über die Lieferkosten, da ich oft Sammelbestellungen mit meinen Klassenkameraden machte. Ich schätze diese auf ca 20€.

Die gesamten Materialien kosteten mich also 91,08€.

Neben den Materialkosten wurde im Rahmen des Projekts auch die benötigte Arbeitszeit dokumentiert und bewertet. Ziel dieser Erfassung war es, ein realistisches Bild der Gesamtkosten zu erhalten, wie sie auch im späteren Berufsleben entstehen würden – auch wenn in unserem Fall natürlich keine tatsächlichen Löhne oder Honorare bezahlt wurden.

Für die Umsetzung meines Projekts investierte ich insgesamt 174 Stunden in Planung, Entwicklung, Schaltungsdesign, Aufbau und Test der Hardware sowie in die Programmierung der Software. Bei einem fiktiven Stundensatz von 40 € ergibt sich daraus ein Betrag von 6.960 €.

Zusätzlich kamen noch einmal 20 Stunden für das Erstellen der Projektdokumentation hinzu, was weiteren 800 € entspricht.

Auch die Beratungsstunden, die ich im Verlauf des Projekts in Anspruch nahm, wurden festgehalten. Diese ergaben sich aus der zuvor gezeigten Tabelle und summierten sich auf insgesamt 5 Stunden, was bei einem fiktiven Stundensatz von 25 € pro Viertelstunde einem Gesamtbetrag von 500 € entspricht.

Rechnet man alle fiktiven Lohnkosten zusammen, ergibt sich ein Gesamtwert von 8.260 € für die geleistete Arbeitszeit und Unterstützung. Zusammengerechnet mit den Materialpreisen kostet das Projekt also etwa 8.351,08€.

8 Fazit

8.1 Selbstreflektion

Während des Projekts konnte ich mich größtenteils gut an die wöchentlichen Berichte und den Zeitplan halten. Diese strukturierte Arbeitsweise half mir, die Übersicht zu behalten und die Aufgaben termingerecht abzuschließen. Besonders viel Freude bereitete mir das Zeichnen der Platine sowie die Entwicklung des Gehäuses in Fusion 360. Hier konnte ich meine Kreativität einbringen und meine technischen Stärken gut nutzen. Auch beim Lösen technischer Probleme, wie etwa dem optischen Aufbau der Anzeige oder dem Verbessern der Platine, konnte ich meine Stärken im praktischen Arbeiten und Konstruieren unter Beweis stellen.

Weniger leicht fiel mir hingegen die Arbeit an der Software. Vor allem das Schreiben von Code ohne vorgefertigte Bibliotheken war für mich herausfordernd und hat mir nicht so viel Spaß gemacht wie die Hardwareentwicklung. Diese Schwäche zeigte sich besonders bei dem Versuch, die Display-Ansteuerung selbst umzusetzen, was mir letztlich nicht gelungen ist. Dennoch sehe ich auch diese Erfahrung als wertvoll an, da sie mir geholfen hat, meine Grenzen zu erkennen und zu lernen, wann es sinnvoll ist, auf bestehende Lösungen zurückzugreifen. Alles in allem konnte ich durch das Projekt nicht nur meine fachlichen Kompetenzen erweitern, sondern auch besser einschätzen, wo meine Stärken liegen.

8.2 Schlussfolgerung

Das Projekt war für mich eine wertvolle und lehrreiche Erfahrung. Ich konnte mein Wissen in den Bereichen Elektronik, Konstruktion und Softwareentwicklung gezielt anwenden und erweitern. Auch wenn nicht alles auf Anhieb funktioniert hat, konnte ich durch Ausprobieren, Rücksprache mit Lehrpersonen und gezieltes Nachbessern funktionierende Lösungen finden. Besonders positiv war, dass ich alle wesentlichen Projektziele termingerecht erreichen konnte und am Ende ein funktionierendes Gesamtsystem entstanden ist.

Rückblickend hat mir das Projekt nicht nur fachlich, sondern auch persönlich viel gebracht. Ich habe gelernt, mit Problemen umzugehen, dranzubleiben und pragmatische Entscheidungen zu treffen. Das Projekt hat mir gezeigt, in welchen Bereichen ich besonders interessiert und motiviert bin und wo ich mich künftig noch weiter verbessern möchte. Insgesamt blicke ich stolz auf die geleistete Arbeit zurück und sehe das Projekt als wichtigen Schritt in meiner technischen Ausbildung.

9 Anhänge

9.1 Projektantrag

Projektantrag

Seite 1/2

Ansuchen für ein Maturaprojekt, Fassung September 2024

Projekttitel:	SnowLens	
Projektleiter:	Alber Lorenz	
Klasse und Schuljahr:	5. BEL	Schuljahr 2024 / 2025

Logo des Projektes:

SnowLens

Kurze Beschreibung des Projektes:

Das Ziel ist es, einen Ski Tracker zu bauen. Er soll die Geschwindigkeit mit einem GPS Sensor messen. Zusätzlich sollte noch die Temperatur gemessen werden. Diese Werte sollen zusätzlich zur Uhrzeit und zum Batteriestand auf einem transparenten Display in der Skibrille ausgegeben werden.
Als Controller wird ein ATMEGA324 mit Arduino-Framework verwendet. Display: 1,5" OLED mit SPI

Benotungsrelevante Meilensteine des Projektes:

Fertige Schaltung - 18.10.2024
Platine bei JLC und Bauteile bestellt - 19.11.2024
Platine fertig bestückt, Bereit zum Testen - 19.12.2024

"Hello World" auf Display funktioniert - 10.02.2025
GPS-Koordinaten sind in VAriablen auslesbar, Temperatur ebenfalls - 06.03.2025
Fertige Software: Anzeige Speed + Temp + BattLevel auf Display - 27.03.2025
Einbau Platine in fertiges Gehäuse: PW:29

Projektantrag

Seite 2/2

Anhänge:	☐ JA ( ) Seiten	☒ NEIN
Projekt genehmigt:	☒ JA	☐ NEIN
Maximal erreichbare Note bei 100% Funktionalität: (unter Berücksichtigung der unten genannten Präzisierungen)	10 (Zehn)	

Präzisierungen des PAG:

- Display ohne Library, sonst - 2 Punkte
- Der Rest: Note je nach Fertigstellung

PAG 1:Martin De Tomaso

PAG 2: Ivan Huber

OK

Der Projektleiter

9.2 Pflichtenheft

Pflichtenheft

SnowLens

Projekttitel:	SnowLens
Projektleiter:	Alber Lorenz

Beschreibung des Projekts:

Das Ziel ist es, ein Anzeigesystem für eine Skibrille zu bauen. Es soll die Geschwindigkeit mit einem GPS Sensor messen. Zusätzlich sollte noch die Temperatur gemessen werden. Diese Werte sollen zusätzlich zur Uhrzeit und zum Batteriestand auf einem transparenten Display in der Skibrille ausgegeben werden.

PSU: Die Spannungsversorgung ist eine 3,7V Flat Lipo Batterie. Sie wird über einen Laderegler mit USB-C aufgeladen. Ein Step Up Converter bringt die Spannung auf 5V. Mit einem Schalter kann die Spannungsversorgung und somit die Platine ein- und ausgeschaltet werden.

µController: Als Controller verwende ich den ATMEGA324 mit einem Arduino-Framework. Von der Batterie führt ein Abgriff zu einem Eingang, sodass der Batteriestand ausgewertet werden kann.

Sensoren: Ich verwende einen GPS Sensor. Anhand seiner Koordinaten und deren Veränderung in einem bestimmten Zeitraum kann ich die Geschwindigkeit bestimmen. Mit einem Temperatursensor messe ich außerdem noch die Außentemperatur.

Display: Das Display ist ein 1,5" Oled Display. Es hat eine I2C Schnittstelle und wird ohne Verwendung einer Library angesteuert.

Gehäuse: Die Platine und sämtliche anderen Komponenten, bis auf den Temperatursensor und das Display werden während des Skifahrens in einer Box im Rucksack aufbewahrt. Das Display an der Skibrille wird mit einem langen Kabel an die Box angeschlossen. Der Temperatursensor wird auch mit einem längeren Kabel aus dem Rucksack geführt

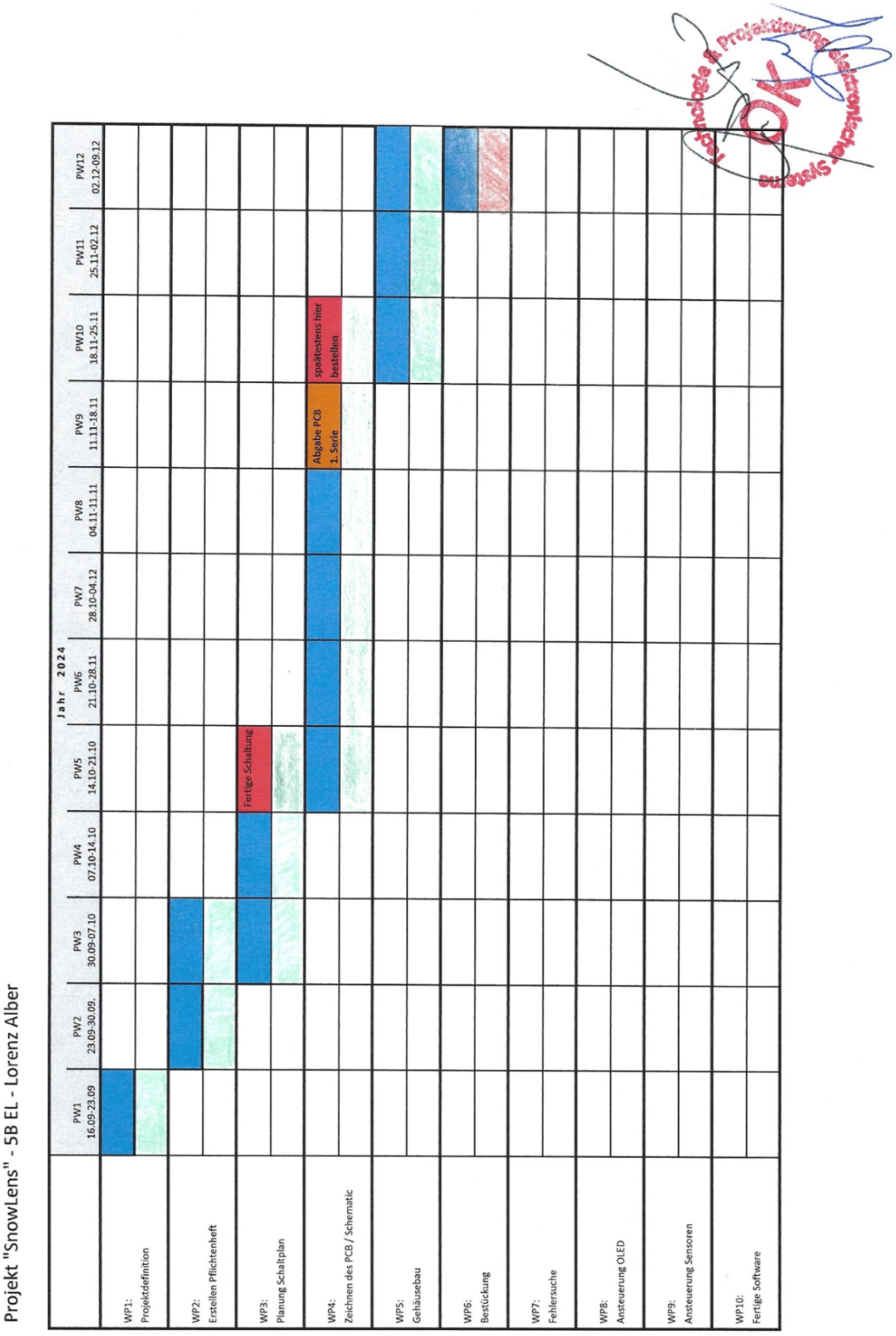
PAG 1:Martin De Tomaso

PAG 2: Ivan Huber

OK

Der Projektleiter

9.3 Gantt- Diagramm



Projekt "SnowLens" - 5B EL - Lorenz Alber

	Jahr 2023			Jahr 2024								
	PW13 09.12-16.12	PW14 16.12-23.12	PW15 23.12-30.12	PW16 31.12-06.01	PW17 06.01-13.01	PW18 13.01-20.01	PW19 20.01-27.01	PW20 27.01-03.02	PW21 03.02-10.02	PW22 10.02-17.02	PW23 17.02-24.02	PW24 24.02-03.03
WP1: Projektdefinition		Evt. Bestllung 2. Serie										
WP2: Erstellen Pflichtenheft												
WP3: Planung Schaltplan												
WP4: Zeichnen des PCB / Schematic												
WP5: Gehäusebau												
WP6: Bestückung		Platiene bestückt										
WP7: Fehlersuche												
WP8: Ansteuerung OLED										Display geht ea.	Display geht on	
WP9: Ansteuerung Sensoren												
WP10: Fertige Software												

Technologie & Projektsteuerung
elektronischer Stempel
OK

SnowLens

TFO MERAN
TECHNOLOGISCHE FACHOBERSCHULE

rojekt "SnowLens" - 5B EL - Lorenz Alber

ABGABE

	Jahr 2024											
	PW25 03.03-10.03	PW26 10.03-17.03	PW27 17.03-24.03	PW28 24.03-31.03	PW29 31.03-06.04	PW30 06.04-13.04	PW31 10.04-16.04	PW32 24.04-30.05	PW33 01.05-07.05	PW34 08.05-14.05	PW35 15.05-21.05	PW37 22.05-28.05
/P1: rojektdefinition												
/P2: stellen Pflichtenheft												
/P3: lanung Schaltplan												
/P4: eichnen des PCB / Schematic												
/P5: ehäusebau												
/P6: estückung												
/P7: ehlersuche												
/P8: nsteuerung OLED												
/P9: nsteuerung Sensoren	GPS-Auswertung											
/P10: rtige Software				Fertige Software								



9.4 Wochenbericht

Wöchentlicher Projektbericht

SnowLens

Projekttitel:	SnowLens		
Projektleiter:	Lorenz Alber		
Projektwoche / Datum:	PW: 11	25.11.2024	02.12.2024

Kurzbericht der erledigten Arbeiten in der Projektwoche:

Ich teste das transparente OLED Display mit der Ansteuerung über I2C. Nach einigen frustrierenden Stunden funktionierte die Ausgabe eines Textes auf dem Bildschirm. Jedoch kommt es immer wieder wegen der schlechten Verkabelung des Steckbretts zu Fehler bei der Kommunikation zwischen Display und Arduino und somit stört das Display oft. Mit einer Platine wird dieses Problem jedoch gelöst, da stabile Verbindungen zum Einsatz kommen. Außerdem begann ich mit dem Zeichnen des Gehäuses auf Fusion.

Im Zeitplan laut GANTT-Diagramm: ☒ JA ☐ NEIN

Wenn NEIN, welche Working Packages (WP) liegen in Verzug?

Alles planmäßig.

Vorgeschlagene Maßnahmen, um den Zeitplan wieder aufzuholen:

Keine notwendig.

Wurden die Maßnahmen aus letzter Woche planmäßig umgesetzt? ☐ JA ☐ NEIN

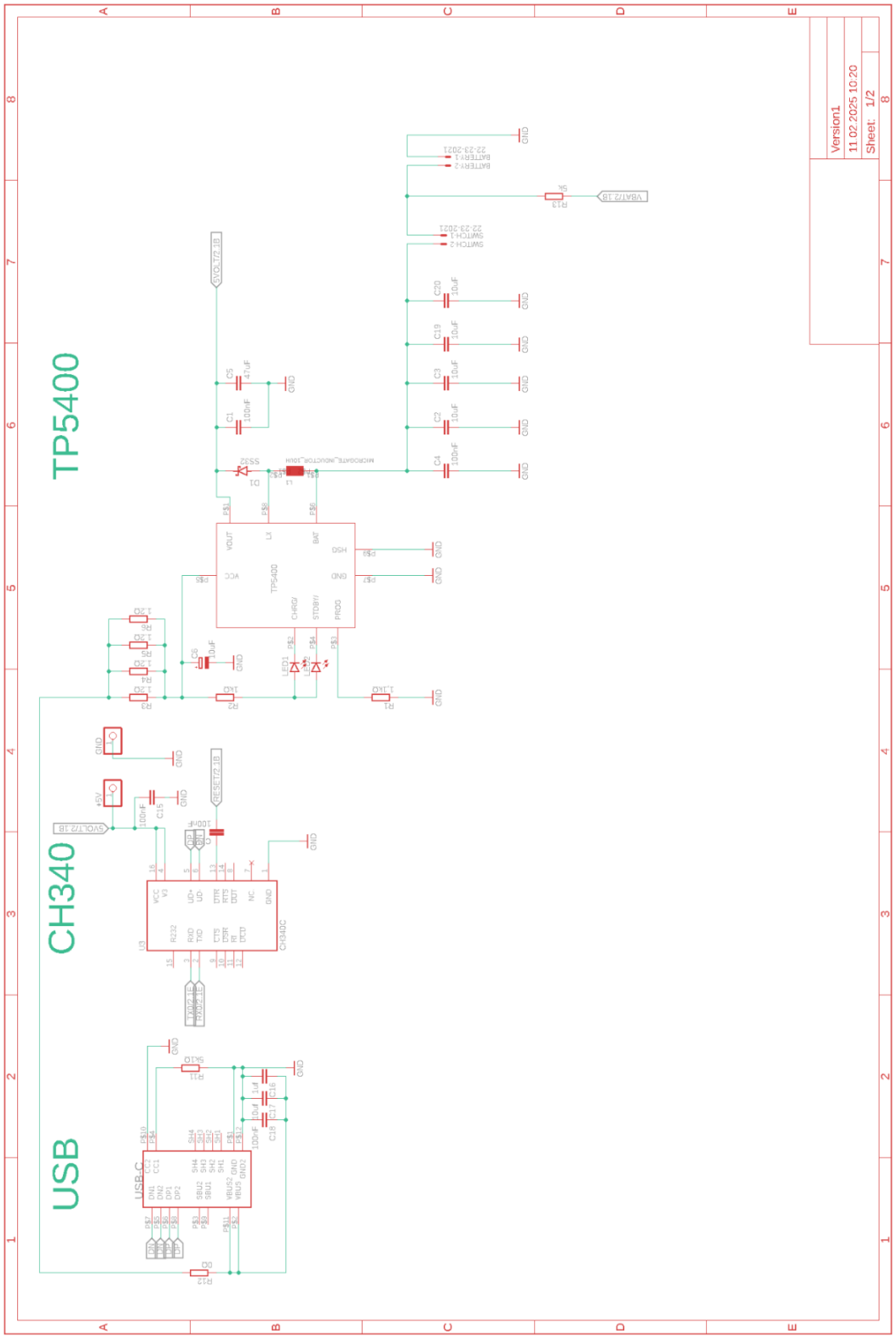
Kurze Zusammenfassung des Zeitaufwandes der Projektwoche:

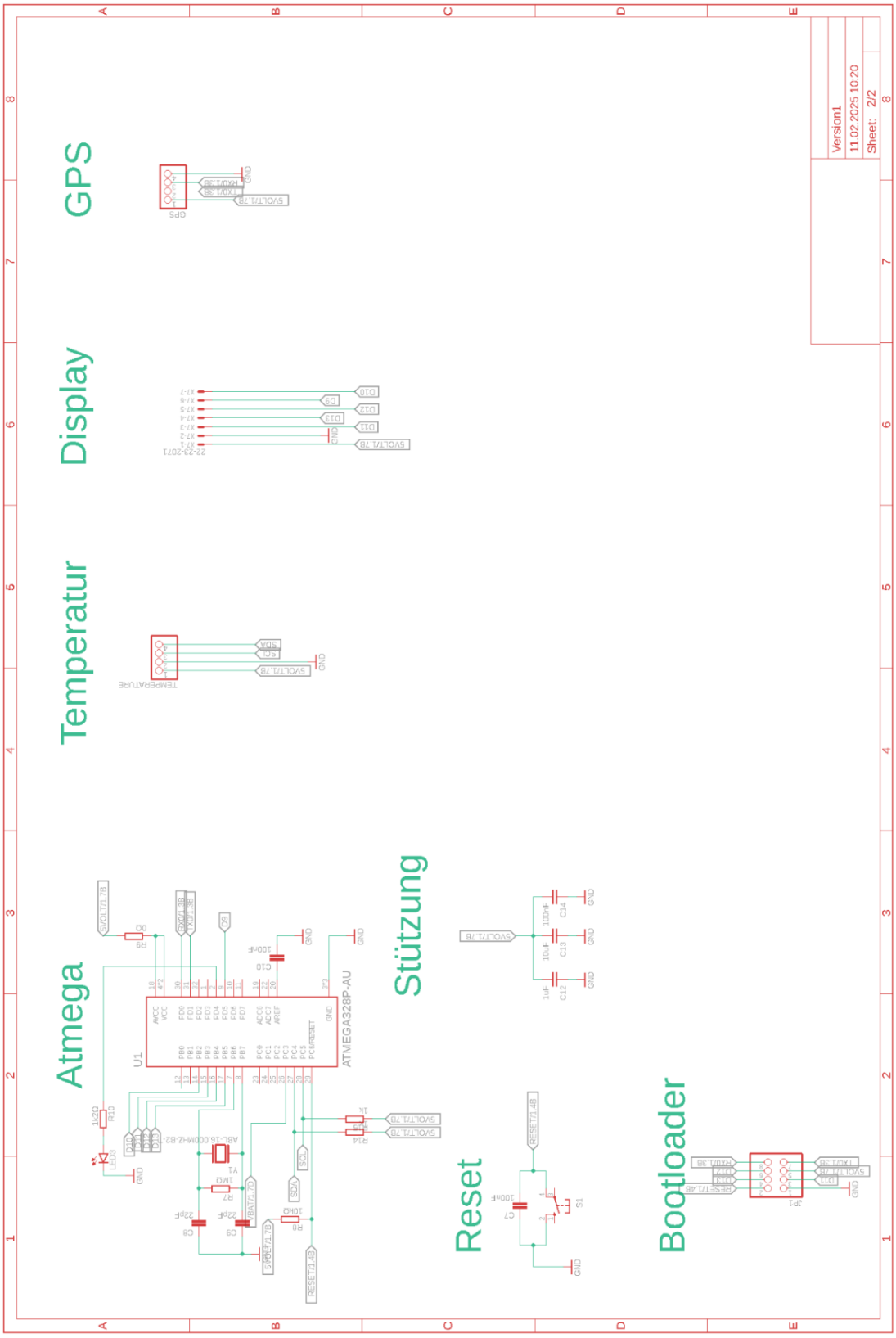
Stunden in der Schule (Labor)	5	Stunden
Stunden im offenen Labor	0	Stunden
Stunden Privat / Zuhause	3	Stunden
Gesamtstundenzahl	8	Stunden

Datum: 28.11.2024

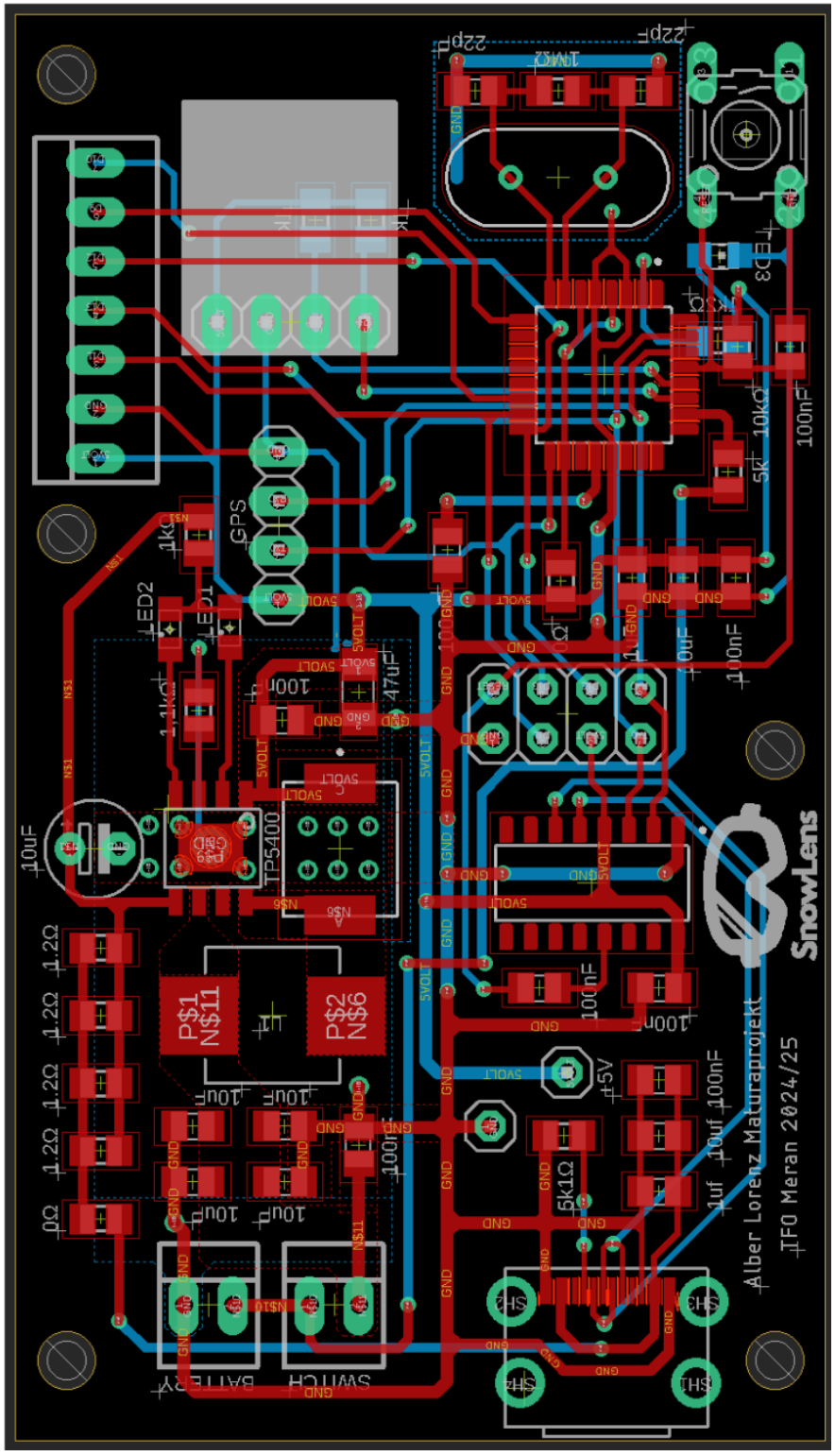
Unterschrift verantwortlicher Projektleiter:

9.5 Schematik




GPS
Display
Temperatur
Atmega
Stützung
Reset
Bootloader
Version1
11.02.2025 10:20
Sheet: 2/2
ATMEGA328P-AU
U1

9.6 Board


SnowLens
Alber Lorenz Maturaprojekt
TFO Meran 2024/25
TP5400
GPS
SWITCH
BATTERY

9.7 Software

```
#include <SPI.h>
#include <U8g2lib.h>
#include <Wire.h>
#include <Adafruit_BMP280.h>
#include <TinyGPSPlus.h>
#include <SoftwareSerial.h>

#define OLED_DC 9
#define OLED_RESET 8
#define OLED_CS 10
#define BATTERY_PIN A3
#define RXPin 2
#define TXPin 3
#define LedPin 4

U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI display(U8G2_R3, OLED_CS,
OLED_DC, OLED_RESET);

SoftwareSerial gpsSerial(RXPin, TXPin);
TinyGPSPlus gps;

Adafruit_BMP280 tempSensor;
bool tempSensorWorking = false;

uint8_t getBatteryPercentage(float voltage)
{
    if (voltage >= 4.2) return 100;
    if (voltage >= 4.1) return 90;
    if (voltage >= 4.0) return 80;
    if (voltage >= 3.9) return 70;
    if (voltage >= 3.8) return 60;
    if (voltage >= 3.7) return 50;
    if (voltage >= 3.6) return 40;
    if (voltage >= 3.5) return 30;
    if (voltage >= 3.4) return 20;
    if (voltage >= 3.3) return 10;
    if (voltage >= 3.25) return 5;
    return 0;
}

void setup()
```

```
{
    Serial.begin(9600);
    gpsSerial.begin(9600);
    pinMode(LedPin, OUTPUT);

    display.begin();
    display.enableUTF8Print();

    display.firstPage();
    do {
        display.setFont(u8g2_font_6x10_tf);
        display.drawStr(0, 10, "Initializing...");
        display.drawFrame(0, 0, display.getWidth(),
display.getHeight());
    } while (display.nextPage());

    delay(1000);

    Wire.begin();

    tempSensorWorking = tempSensor.begin(0x76) ||
tempSensor.begin(0x77);

    display.firstPage();
    do {
        display.setFont(u8g2_font_6x10_tf);
        if (tempSensorWorking) {
            display.drawStr(0, 10, "Temp sensor online");
        } else {
            display.drawStr(0, 10, "Temp sensor offline");
        }
    } while (display.nextPage());

    delay(1000);

    display.firstPage();
    do {
        display.setFont(u8g2_font_6x10_tf);
        display.drawStr(0, 10, "GPS Weather Display");
        display.drawStr(0, 25, "Starting up...");
    } while (display.nextPage());

    delay(1000);
```

```
}

void loop()
{
    digitalWrite(LedPin, HIGH);
    bool newData = false;
    for (unsigned long start = millis(); millis() - start < 500;)
    {
        while (gpsSerial.available())
        {
            char c = gpsSerial.read();
            if (gps.encode(c))
                newData = true;
        }
    }

    float batteryVoltage = (analogRead(BATTERY_PIN) / 1023.0) * 5.0;
    uint8_t batteryPercentage =
getBatteryPercentage(batteryVoltage);

    float temperature = 0.0;
    float RealTemp  = 0.0;
    if (tempSensorWorking) {
        temperature = tempSensor.readTemperature();
        RealTemp = temperature - 8;
    }

    display.firstPage();
    do {
        display.setFont(u8g2_font_6x10_tf);

        char buffer[32];
        dtostrf(batteryVoltage, 4, 2, buffer);

        char batteryPctStr[32];
        sprintf(batteryPctStr, "%u%%", batteryPercentage);
        display.drawStr(24, 34, batteryPctStr);

        display.drawStr(4, 94, "   km/h");

        if (gps.time.isValid())
        {
            char timeStr[32];
```

```
            int italyHour = (gps.time.hour() + 2) % 24;
            sprintf(timeStr, "  %02d:%02d:%02d",
                    italyHour, gps.time.minute(),
gps.time.second());
            display.drawStr(0, 22, timeStr);
        }

        if (tempSensorWorking) {
            dtostrf(RealTemp, 4, 1, buffer);
            char tempStr[32];
            sprintf(tempStr, "  %s*C", buffer);
            display.drawStr(4, 46, tempStr);
        } else {
            display.drawStr(0, 46, "Temp: ERROR");
        }

        display.setFont(u8g2_font_10x20_tf);
        if (gps.speed.isValid())
        {
            float speedKmh = gps.speed.kmph();
            dtostrf(speedKmh, 5, 1, buffer);
            char speedStr[32];
            sprintf(speedStr, "%s", buffer);
            display.drawStr(0, 80, speedStr);
        }
        else
        {
            display.drawStr(8, 80, " N.D.");
        }

    } while (display.nextPage());

    delay(100);
}
```

9.8 Quellen

Bild auf Seite 7, Inspiration:
https://i0.wp.com/scifiinterfaces.com/wp-content/uploads/2015/06/ironman_hud01.png?ssl=1

Bild auf Seite 11, Atmega 328P:
https://res.cloudinary.com/rsc/image/upload/b_rgb:FFFFFF,c_pad,dpr_2.625,f_auto,h_214,q_auto,w_380/c_pad,h_214,w_380/F1310270-01?pgw=1

Bild auf Seite 12, GPS Modul:
https://m.media-amazon.com/images/I/512x4cPF9cL.jpg

Bild auf Seite 12, BME280:
https://store.ichibot.id/wp-content/uploads/2024/12/63-6.png

Bild auf Seite 13, Display:
https://m.media-amazon.com/images/I/61eTWPB+QLL.jpg

Bild auf Seite 14, Batterie:
https://thepihut.com/cdn/shop/products/2000mah-3-7v-lipo-battery-the-pi-hut-105187-39650305409219.jpg?crop=center&height=1200&v=1664378081&width=1200

Bild auf Seite 15, TP5400:
https://http2.mlstatic.com/D_NQ_NP_726570-MLB44507125682_012021-O-tp5400-ci-smd-para-carregador-de-bateria-de-litio.webp

Bild auf Seite 16, USB-C Port:
https://res.cloudinary.com/rsc/image/upload/b_rgb:FFFFFF,c_pad,dpr_2.625,f_auto,h_214,q_auto,w_380/c_pad,h_214,w_380/F2153154-01?pgw=1

Bild auf Seite 16, CH340:
https://components101.com/sites/default/files/components/CH340-IC.jpg

Bild auf Seite 39, Optik:
https://m.media-amazon.com/images/I/61MS6sDoYYS._AC_SX679_.jpg

Zur Recherche wurden Anwendungen verwendet: Google-Suche, Chat GPT, Copilot.
Zahlreiche Informationen wurden aus Foren, Portalen und Tutorials entnommen.

9.9 Copyright

Hiermit erkläre ich, Lorenz Alber, dass ich das vorliegende Abschlussprojekt und die zugehörige Dokumentation für das Abschlussprojekt der Technologischen Fachoberschule „Oskar von Miller“ eigenständig und erstellt und ausschließlich die angegebenen Hilfsmittel und Quellen genutzt habe.

Projektleiter - Alber Lorenz

_____

Fachlehrperson - De Tomaso Martin

_____

Fachlehrperson - Huber Ivan

_____

Direktor - Augscheller David

_____